

FastCAE-Fluent 集成示例

（可视化集成与插件拓展集成）

FastCAE 研发小组

2020 年 03 月 25 日

1. 概述

1.1 案例介绍

本案例基于 FastCAE 提供的插件拓展机制与可视化定制功能，针对石油行业中使用旋滤器进行油水分离的特定工况，开发了 Fluent 适配插件，实现了 FastCAE 与 Fluent 的数据交换，以及 Fluent 的计算驱动，形成了这对这一特定工况的仿真分析软件系统，该软件系统与 Fluent 无缝集成，是一套集前后处理与求解计算于一体的专业计算平台。

本案例名称为《Fluent 集成案例》，该案例所有代码可以在网站 <https://pan.baidu.com/s/1o2uJJPNxcurPvoOipdWsDw>（提取码：azp0）处下载。

1.2 集成目标

- 通过 FastCAE2.0 集成框架完成一个典型的以 Fluent 软件为求解器的软件产品的集成工作。
- 通过本案例能够使用户快速了解使用 FastCAE 2.0 进行插件开发与定制开发方式相结合的开发过程，为 Fluent 软件集成与二次开发提供指导。

1.3 环境说明

- 开发环境

- Windows 10
- VS2013 Community(VS 2013 (其他版本也可以))
- Qt 5.4.2-msvc-opengl-x64
- FastCAE 2.0
- Fluent 18.0

1.4 注意事项

- 建议开展本案例学习之前，用户需要有一定 C++基础和 Fluent 使用经验。
- 由于 Fluent 导入 jou 脚本里文件路径不能有中文，所以脚本所使用的网格文件应放在不包含中文的路径中。在 FastCAE 使用过程中应尽量避免代码存放位置及文件存放位置中包括中文路径。
- Fluent 插件开发部分代码设置的边界条件及写出的网格边界条件代码仅适用于类似于本案例的，边界条件只使用到 Fluent 边界条件中 velocity-inlet, outflow, wall 这三种的案例。用户可以按照自己的需求，修改对应边界处的代码来满足具有其他边界要求的 Fluent 集成软件开发。

2. 软件实现效果

本案例最终形成了基于 Fluent，针对特定对象与特定工况的软件系统，该系统的 CFD 仿真计算由 Fluent 内核实现，前后处理功能与流程通过 FastCAE 定制实现。前后处理的个性化流程定制

通过 FastCAE 的可视化定制功能实现，前后处理与 Fluent 的数据交换通过 Fluent 适配插件实现。这套针对于特定对象和特定工况的 CFD 软件固化了分析流程，只需要极少的参数设置就能完成复杂的 CFD 计算，在保证分析精度的前提下，降低了学习成本，提高了仿真分析的效率与可靠性。同时也为其他情境下的复杂 CFD 仿真流程简化提供了可行的解决方案。

在完成软件的开发工作后，用户可以运行软件进行数值计算，我们可以看到，在主界面左侧为控制面板，我们的工程树设置，关联网格，设置边界条件等都可以在工程树上进行操作。右侧主体部分为展示窗体区域，前处理窗口，后处理三维渲染窗口都在这个区域进行显示。在下面相邻的两个区域分别为控制台和进程窗口，控制台用于显示用户的操作流程，进程窗体则主要对求解状态进行显示及控制。完成的软件整体效果如图 1 Fluent 案例集成软件效果所示。

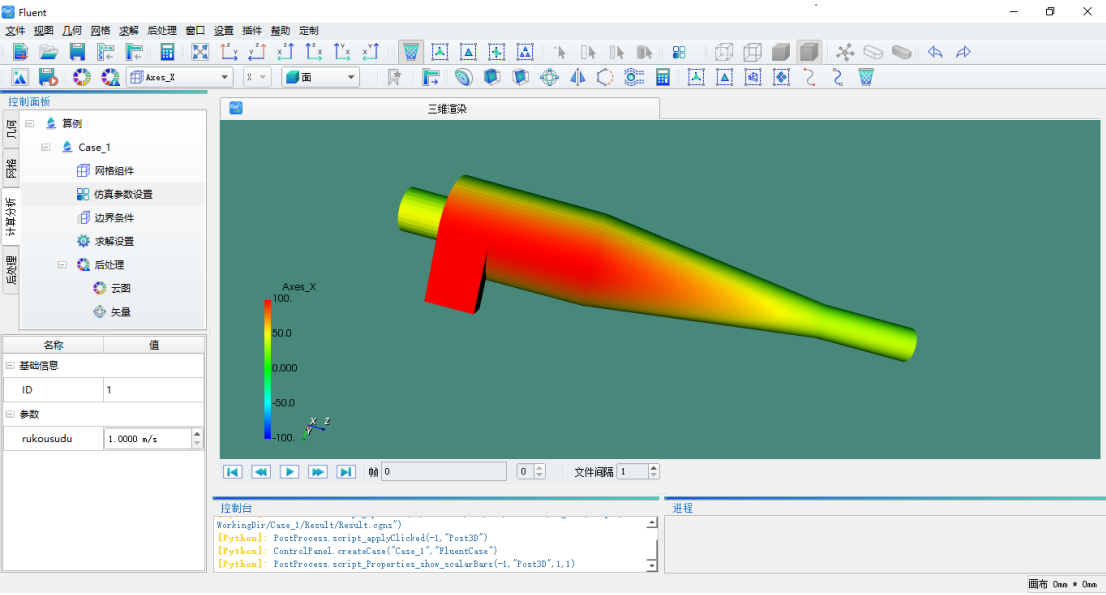


图 1 Fluent 案例集成软件效果

3. 原理说明

本案例的核心集成原理为：通过 FastCAE 平台软件定制界面，并生成 Fluent 运行所需的脚本文件，后台驱动 Fluent 软件进行科学计算并对计算结果进行展示。

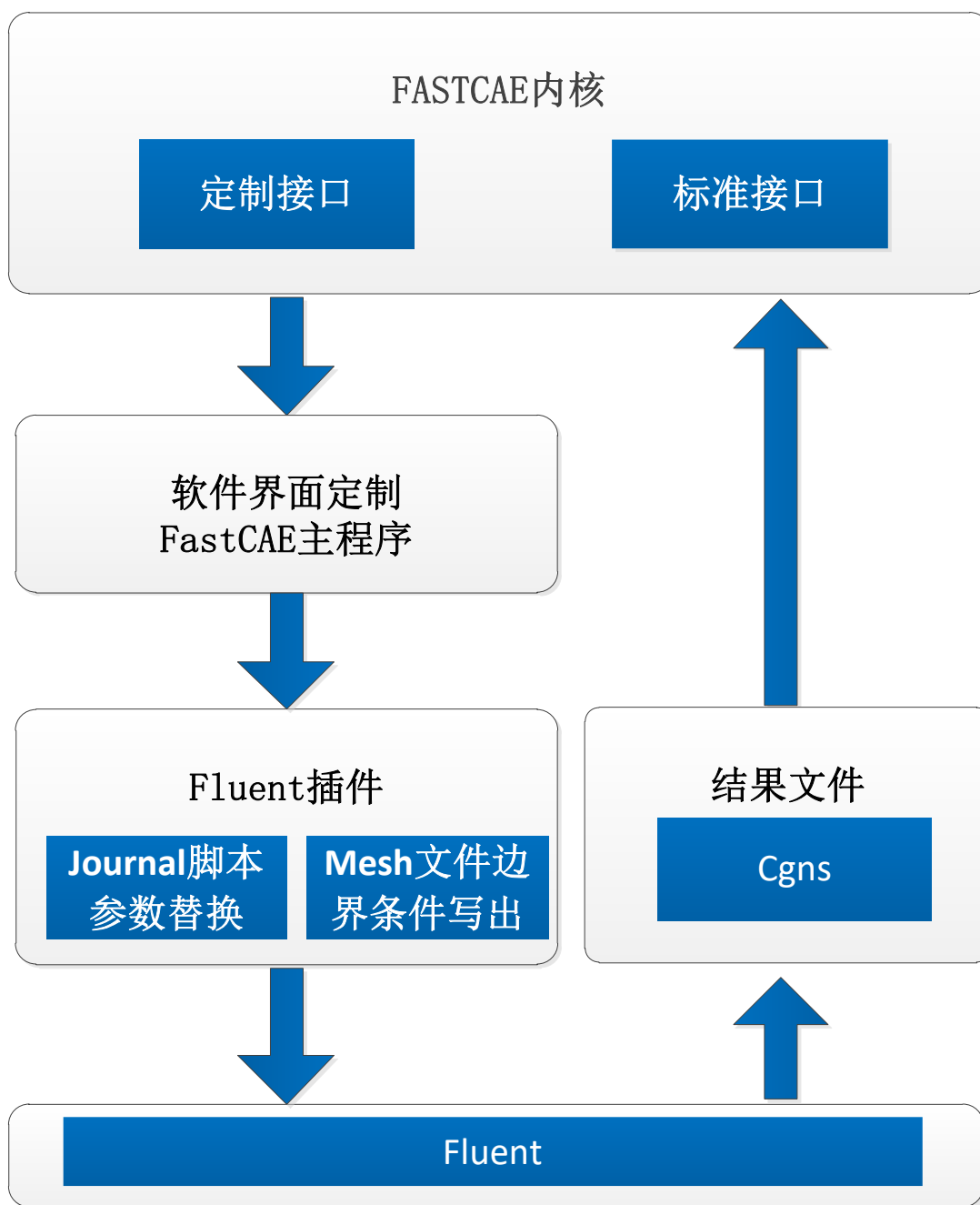


图 2 案例原理图

3.1 Fluent 部分

- Fluent 软件支持通过命令行的方式使用 Jou 脚本进行启动运行。

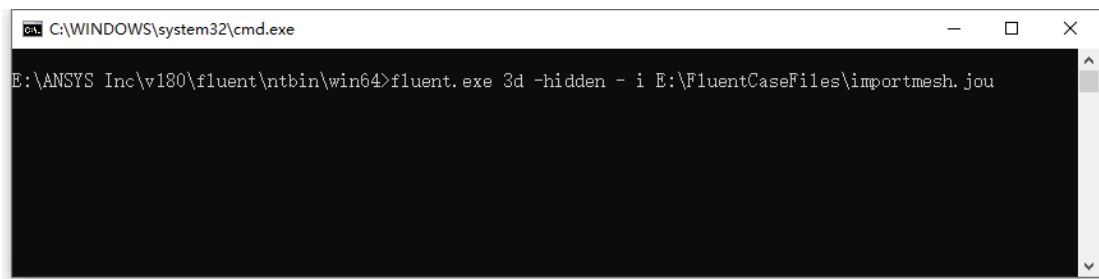


图 3 命令下执行脚本文件

如上图, `fluent.exe` 就会在后台加载 `importmesh.jou` 脚本进行运算。Fluent 命令行参数 `-i journal` 表示读取指定的脚本。`-hidden` 表示以最小化形式运行 `fluent`。`3d` 表示使用的网格文件使用的网格文件是三维的。需要说明的是 `-i journal` 命令行参数时全平台适用的, 而 `-hidden` 命令行参数只支持 windows 平台。

3.2 FastCAE 部分

- 根据 3.2 FastCAE 部分部分内容, 我们可以分析得出 **FastCAE** 部分需要调用 `fluent.exe` 通过传参的形式让 `fluent.exe` 在后台运行, 即我们在 **FastCAE** 部分需要完成 `journal` 脚本内容的设置、显示、及脚本文件中参数的替换的工作。

其中, **FastCAE** 定制化插件部分及软件框架将提供软件参数的设置、网格的导入及展示、边界条件设置、后处理结果展示功能。这部分功能不需要用户重新开发, 只需要进行配置就能够使用

而当前版本 **FastCAE2.0** 没有支持的功能: `msh` 文件边界条件写出功能、`jou` 脚本参数替换功能。这部分功能将由定制化脚本提供。

4. 操作/开发步骤

首先我们需要进行插件开发，将当前 FastCAE2.0 版本不支持的功能通过插件的形式进行开发，通过平台加载插件的形式完成这部分功能的支持。

由于我们开发的插件中具有 jou 脚本参数替换功能，这就需要在完成插件开发后，按照要求对我们计算需要的 jou 脚本进行修改进而形成 jou 脚本模板。

完成 jou 脚本模板边界后，我们需要按照我们软件的要求对软件的界面及功能进行定制。定制完成后，我们就可以开始我们所开发软件的使用旅程了！

4.1 软件定制

在软件定制中，我们先进行通用项的定制，包括：软件名称、公司信息、网格功能、几何功能，及模型树等。

然后进行求解器需要参数的定制，我们将按照需要替换的参数来进行软件参数的定制，这里我们需要替换的参数有：

Viscous Model、Oil Droplet Diameter、Velocity Magnitude、Volume Fraction、Flow Rate Weighting(Up)、Flow Rate Weighting(In)、Iterations、Absolute Criteria。其中按照类型将前六个参数归类为仿真参数，将后两个参数归类为求解参数。

在完成求解器需要的参数定制后，我们需要对后处理文件进行设置。

在完成后续文件设置后，我们需要对求解器进行设置。其中一些设置工作可以穿插进行，上面列举的定制流程是相对通用的一个定制流程，用户可根据自身操作习惯完成所需的定制工作。

下面将详细的对定制过程进行说明。

- 启动 FastCAE 2.0 版本软件加载 PluginCustomizer.dll 插件，开始进行软件常规项定制。

1. 点击【插件】-【插件管理】。

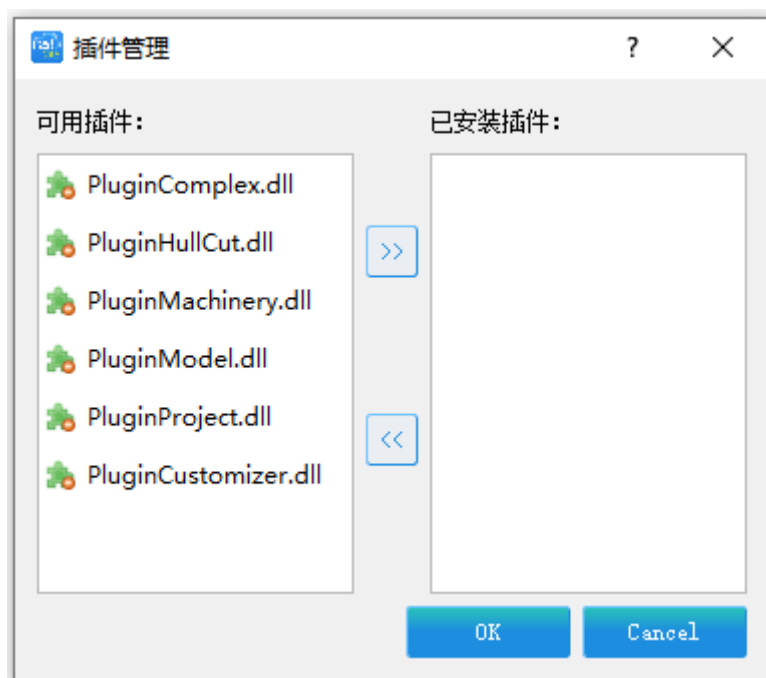


图 4 插件管理

2. 在可用插件列表选中 PluginCustomizer.dll 插件，点击  按键。

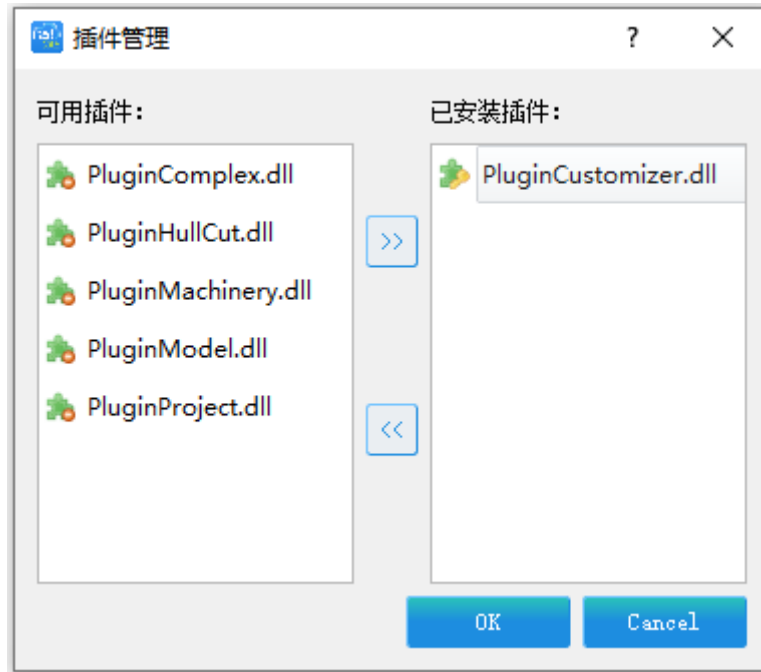


图 5 加载插件

3. 点击  按钮，这样 PluginCustomizer 插件将会被 FastCAE 软件装载。软件菜单栏上将会出现定制菜单项。



4. 点击【定制】-【开始定制】，软件标题栏上名称后面会有“---定制中”字样，表示此时我们处于定制模式。



5. 点击【帮助】-【关于】，软件将弹出软件基本参数设置界面，这里面我们按照下图输入我们想要填入的软件基本信息。然后点击  按钮，完成软件基本信息的录入。



图 6 软件基本参数设置

6. 由于我们 Fluent 案例不需要网格剖分、网格质量检查和创建组件功能, 为保证最终软件最简化, 我们定制去掉这部分功能。点击【网格】, 在菜单中将【面网格划分】、【体网格划分】、【网格质量检查】、【创建网格组件】菜单前面的勾去掉。

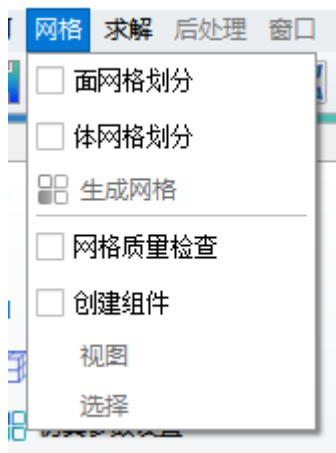


图 7 网格功能设置

7. 我们的 Fluent 案例导入的网格类型为.msh 文件, 为保证最终

软件最简化，我们定制软件只需要支持.msh 格式网格文件。点击【文件】-【导入网格】，软件将弹出软件导入网格项配置界面，在界面中我们只保留【Fluent(*.msh)】项，将【CGNS(*.cgns)】、【Gambit(*.neu)】、【STL(*.stl)】、【Tecplot(*.dat)】、【VTK(*.vtk)】、【Abaqus(*.inp)】项的勾取消掉。然后点击  按钮完成导入网格功能更的定制。

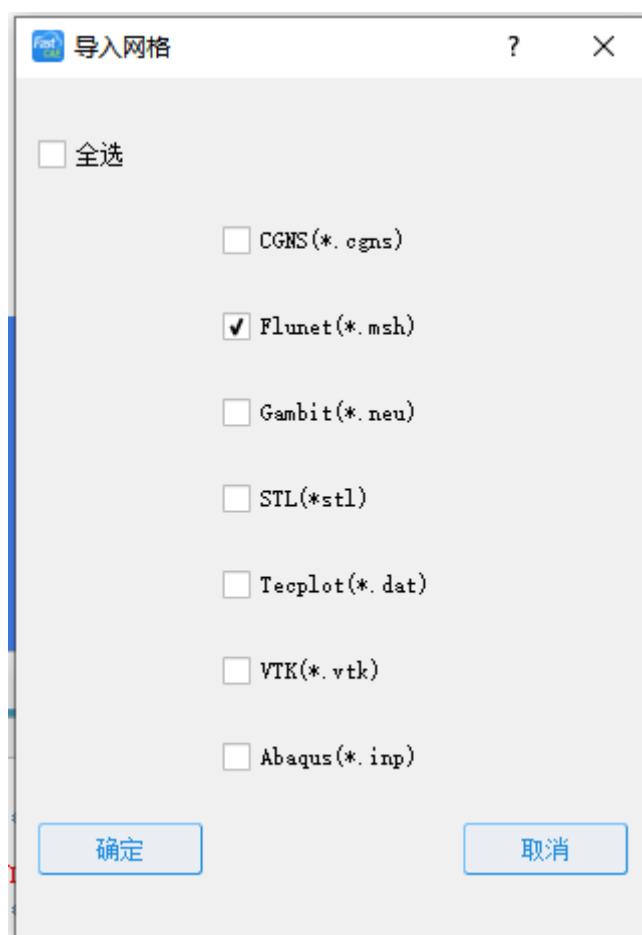


图8 导入网格格式设置

8. 我们的案例不需要导出网格功能，为保证最终软件最简化，我们定制将该部分功能去掉。点击【文件】-【导出网格】，软件将弹出导出网格功能定制界面，我们将全选前面的勾选的勾去掉，确保勾选选项处于反选状态，点击  按钮，完成导出网格功能的定制。

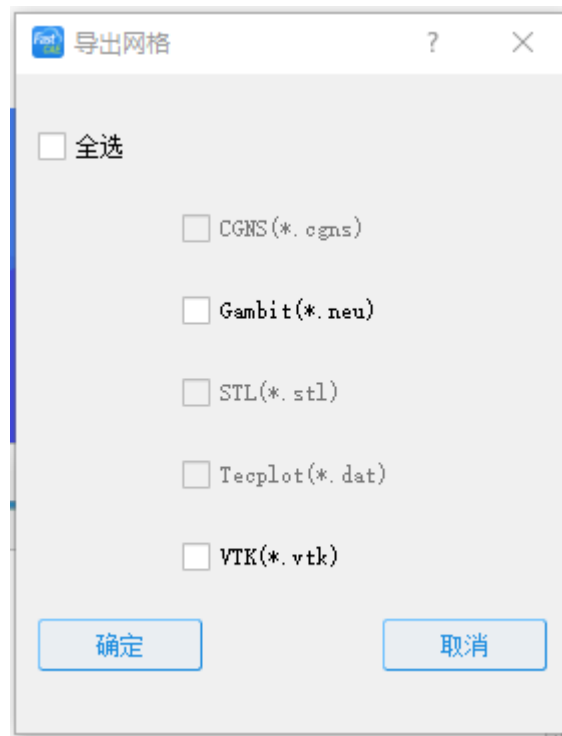


图 9 导出网格格式设置

9. 下面我们进行模型树的定制，将控制面板切换到计算分析分页。



图 10 计算分析页

10. 鼠标左键选中算例节点，点击鼠标右键，点击弹出右键菜单的【添加算例】菜单项。

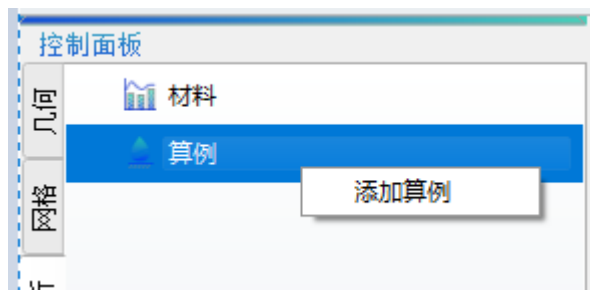


图 11 添加算例

11. 软件将弹出添加算例对话框，我们按照下图填写算例的中文名

称和英文名称。点击  按钮完成算例名称信息的录入。

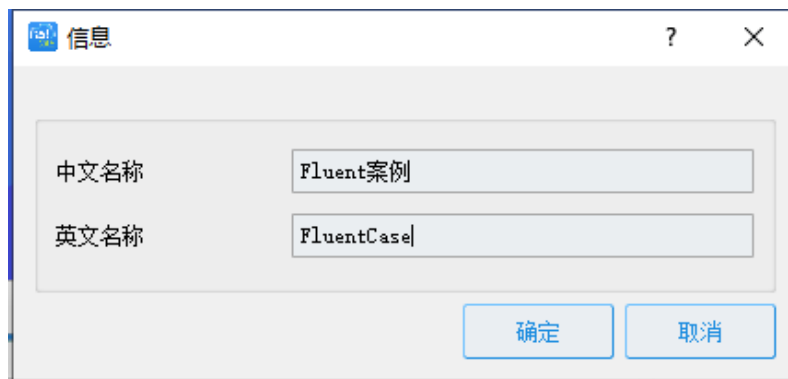


图 12 添加算例信息

12. 此时，【控制面板】-【算例】节点下面将会有一颗用于定制的算例树。

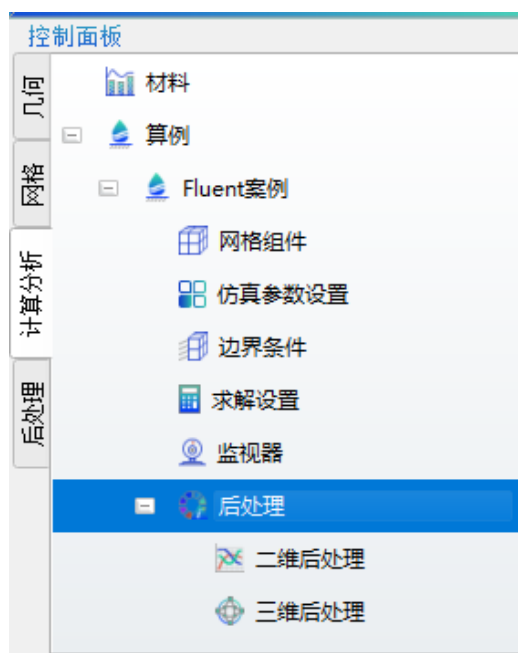


图 13 算例树

13. 鼠标左键双击算例树下的【仿真参数设置】节点，软件将弹出仿真参数设置界面。

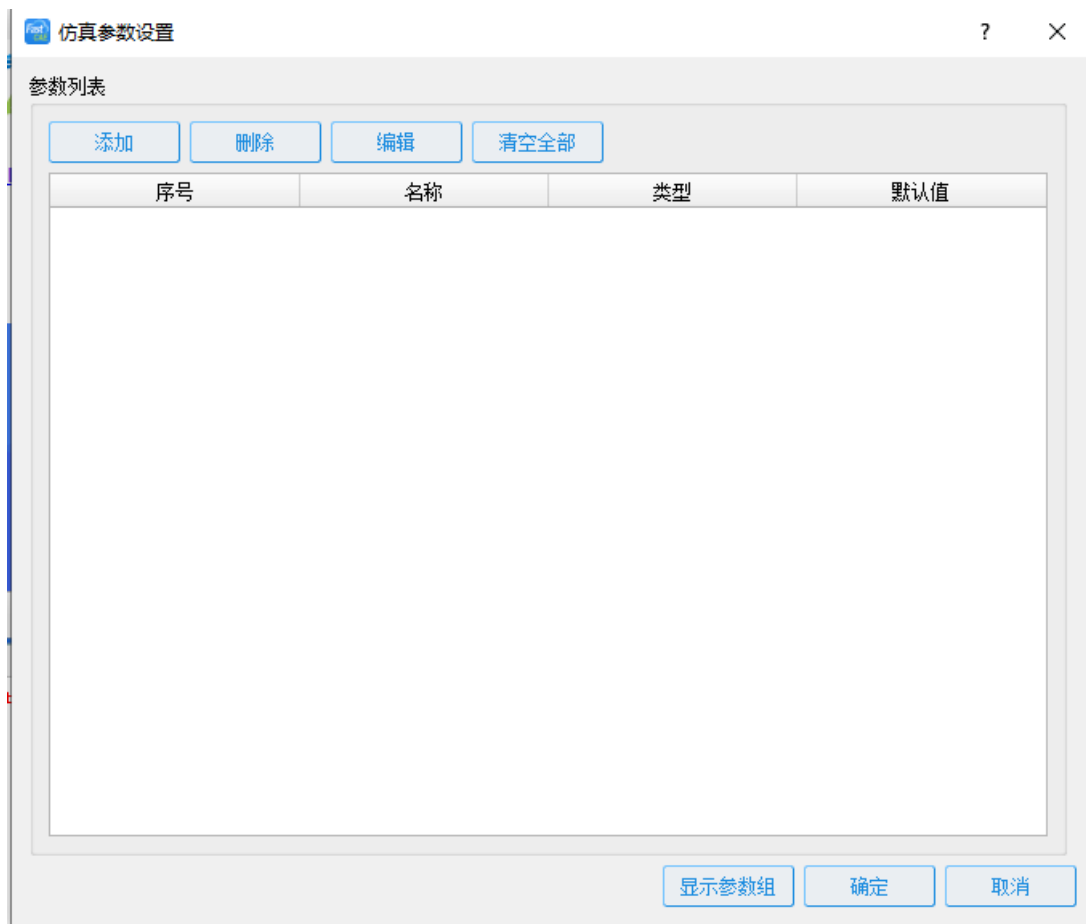


图 14 仿真参数设置

14. 鼠标左键点击仿真参数设置界面中的添加按钮，弹出添加参数类型选择的菜单。

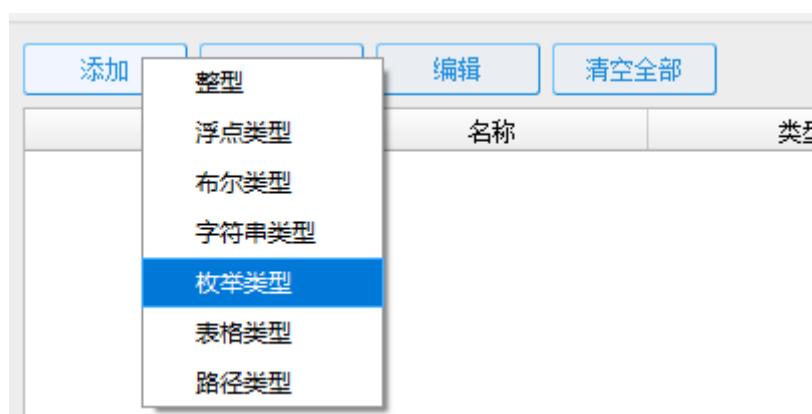


图 15 添加枚举参数

15. 在弹出的添加参数类型选择菜单中，鼠标左键点击【枚举类型】菜单项，软件将弹出添加枚举参数对话框。在枚举参数对话框名称处

填入 Viscous Model，即该枚举项的名称。

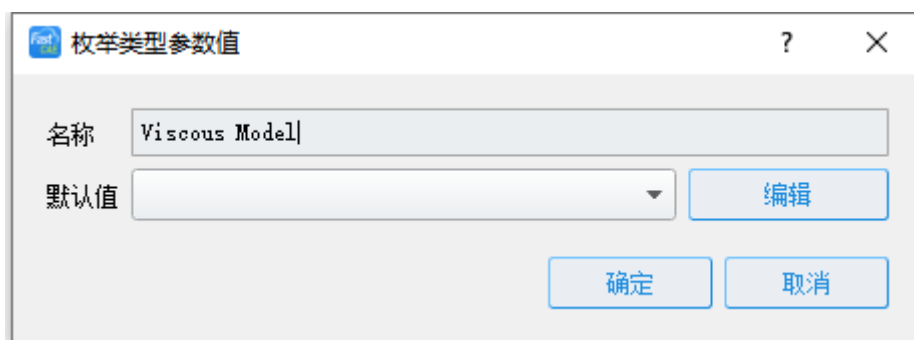


图 16 设定枚举参数名称

16. 点击添加枚举类型参数对话框中的  按钮，软件将弹出枚举值编辑对话框。



图 17 编辑枚举参数值

17. 点击枚举值编辑对话框中的  按钮，软件将弹出添加枚举单个值的对话框，在对话框中填入如下图的参数后点击  按钮，完成枚举项【K-omega(2 eqn)】项的添加。**注意：**涉及到中英文具有差异的符号进行输入的时候，要使用英文符号进行输

入。这里的括号如果使用中文括号，**Fluent** 将不识别。



图 18 添加枚举项 1

18. 点击枚举值编辑对话框中的 **添加** 按钮，软件将弹出添加枚举单个值的对话框，在对话框中填入如下图的枚举参数值后点击 **确定** 按钮，完成枚举项【K-epsilon(2 eqn)】项的添加。

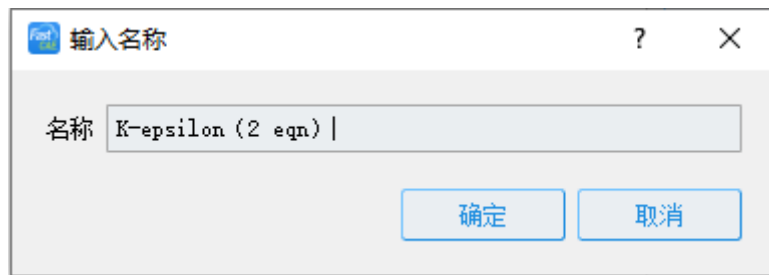


图 19 添加枚举项 2

19. 点击枚举值编辑对话框中的 **添加** 按钮，软件将弹出添加枚举单个值的对话框，在对话框中填入如下图的枚举参数值后点击 **确定** 按钮，完成枚举项【Reynolds Stress】项的添加。

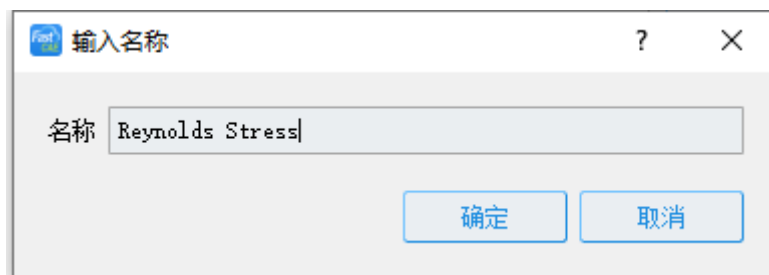


图 20 添加枚举项 3

20. 此时枚举值编辑对话框的界面如下图所示。点击界面中的 **确定** 按钮，完成枚举项的添加工作。



图 21 添加枚举参数效果

21. 此时的枚举参数对话框如下图所示，击界面中的  按钮，完成枚举参数的添加工作。

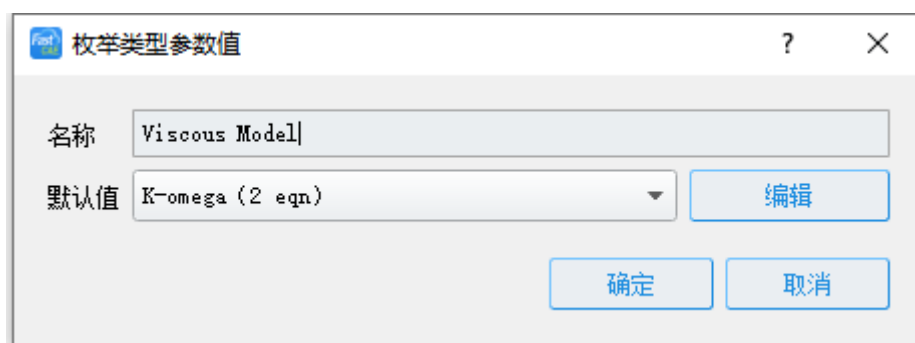


图 22 设置默认枚举值

22. 鼠标左键点击仿真参数设置界面中的  按钮，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击  按钮，完成 Oil Droplet Diameter 浮点参数的添加。

名称	Oil Droplet Diameter
单位	M
浮点精度	6
默认值	0.0005
最小值	0.00001
最大值	0.999

确定 取消

图 23 添加浮点参数 1

23. 鼠标左键点击仿真参数设置界面中的 **添加** 按键，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击 **确定** 按键，完成 Velocity Magnitude 浮点参数的添加。

名称	Velocity Magnitude
单位	M/S
浮点精度	1
默认值	10
最小值	0.1
最大值	1000

确定 取消

图 24 添加浮点参数 2

24. 鼠标左键点击仿真参数设置界面中的 **添加** 按键，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将

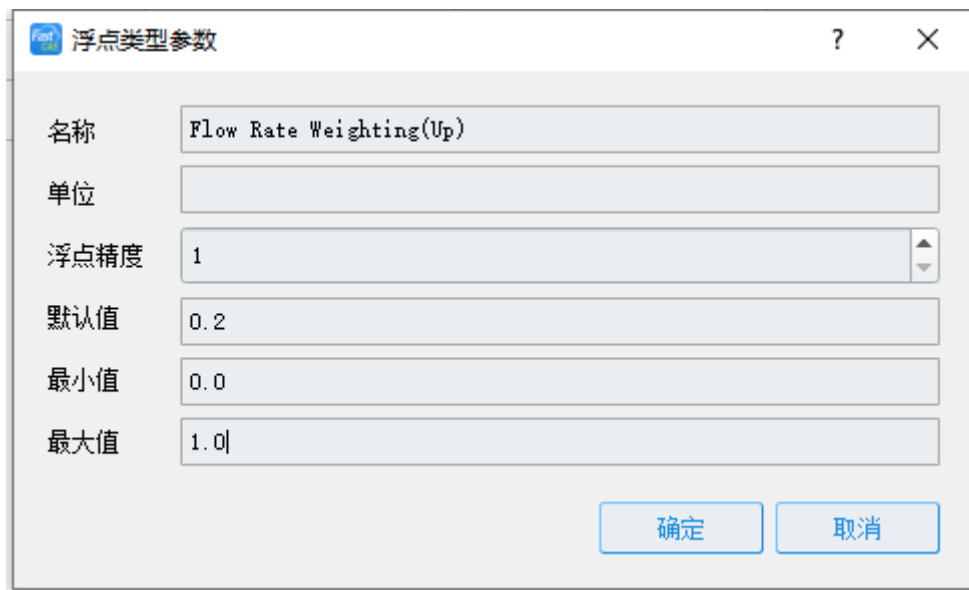
弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击 确定 按键，完成 Volume Fraction 浮点参数的添加。

名称	Volume Fraction
单位	
浮点精度	2
默认值	0.02
最小值	0.0
最大值	1.0

确定 取消

图 25 添加浮点参数 3

25. 鼠标左键点击仿真参数设置界面中的 添加 按键，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击 确定 按键，完成 Flow Rate Weighting(Up)浮点参数的添加。

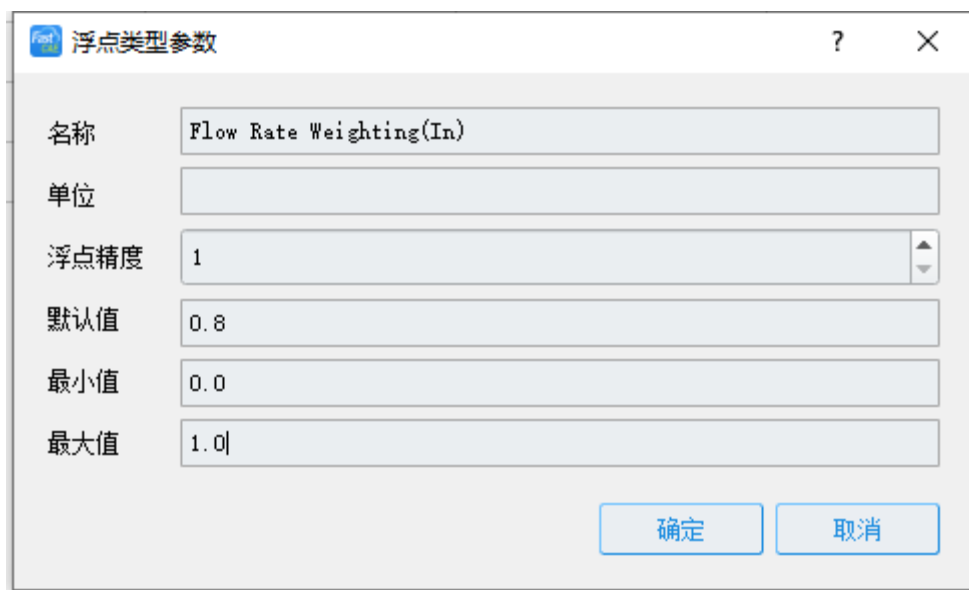


名称	Flow Rate Weighting(Up)
单位	
浮点精度	1
默认值	0.2
最小值	0.0
最大值	1.0

确定 取消

图 26 添加浮点参数 4

26. 鼠标左键点击仿真参数设置界面中的 **添加** 按钮，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击 **确定** 按钮，完成 Flow Rate Weighting(In)浮点参数的添加。



名称	Flow Rate Weighting(In)
单位	
浮点精度	1
默认值	0.8
最小值	0.0
最大值	1.0

确定 取消

图 27 添加浮点参数 5

27. 此时我们仿真参数设置界面如下图所示，点击  按键后，完成仿真参数的设置。

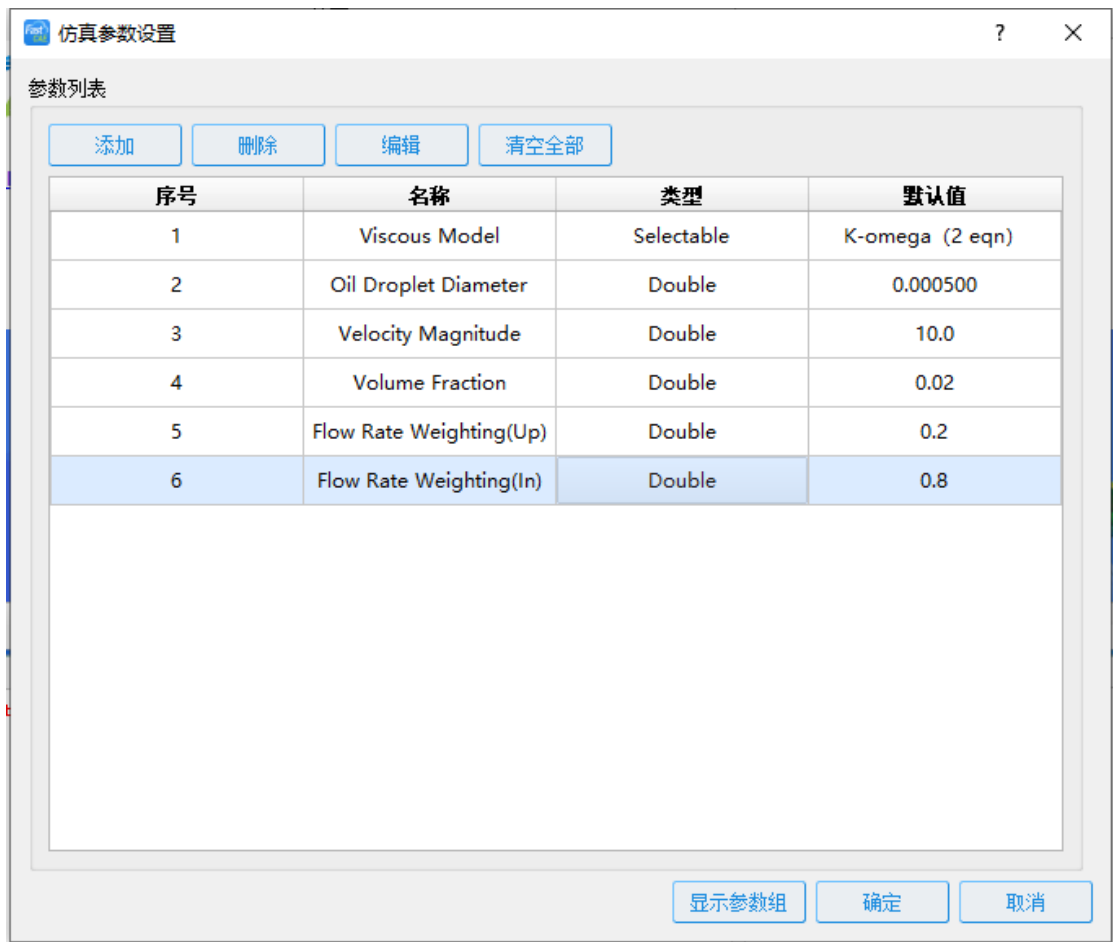


图 28 仿真参数设置完成

28. 鼠标左键双击 **【控制面板】 - 【算例】** 节点下的 **【边界条件】** 节点，软件将弹出边界条件编辑对话框，由于我们的 **Fluent** 算例只使用到 **【入口】**、**【出口】**、**【壁面】** 三种边界，因此我们按照下图所示对软件边界条件进行勾选定制。然后点击  按键，完成边界条件定制。



图 29 边界条件设置

29. 鼠标左键双击【控制面板】-【算例】节点下的【求解设置】节点，软件将弹出求解参数设置界面。

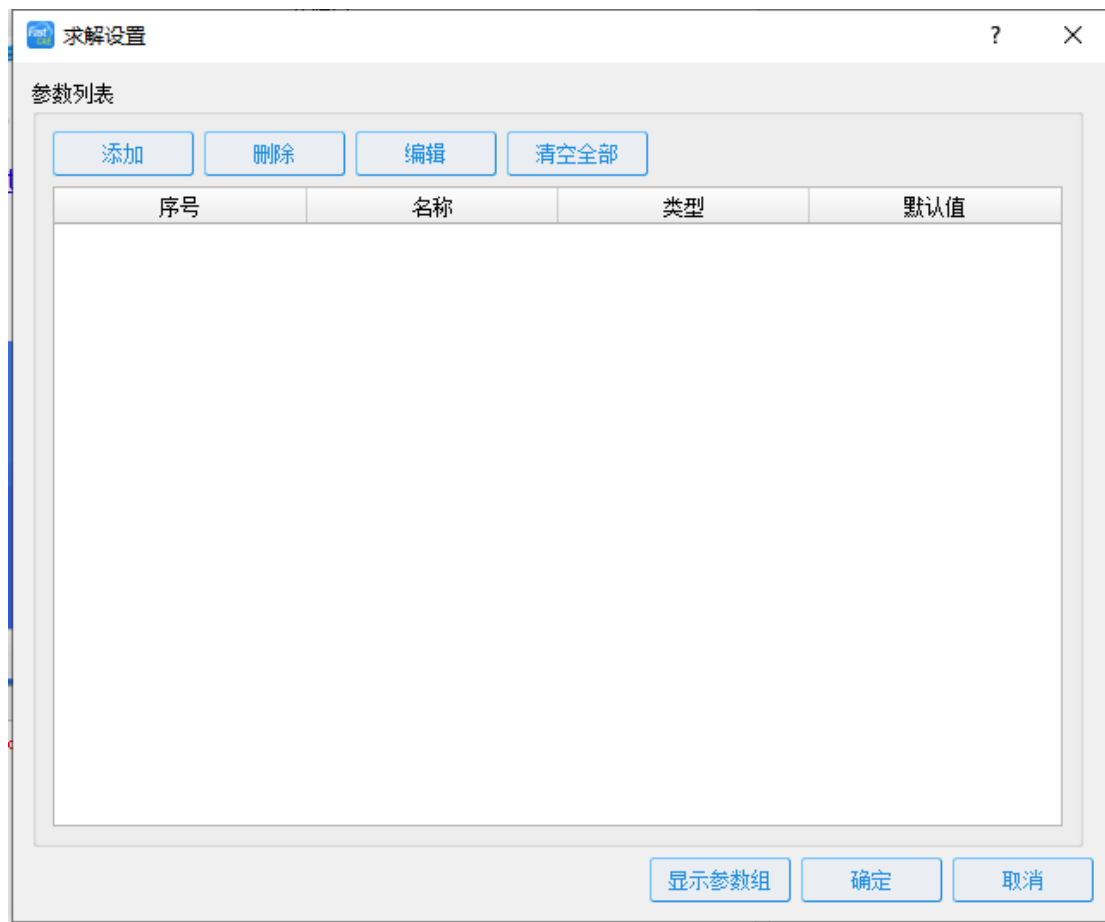


图 30 求解参数设置

30. 鼠标左键点击求解参数设置界面中的 **添加** 按键，弹出添加参数类型选择的菜单。

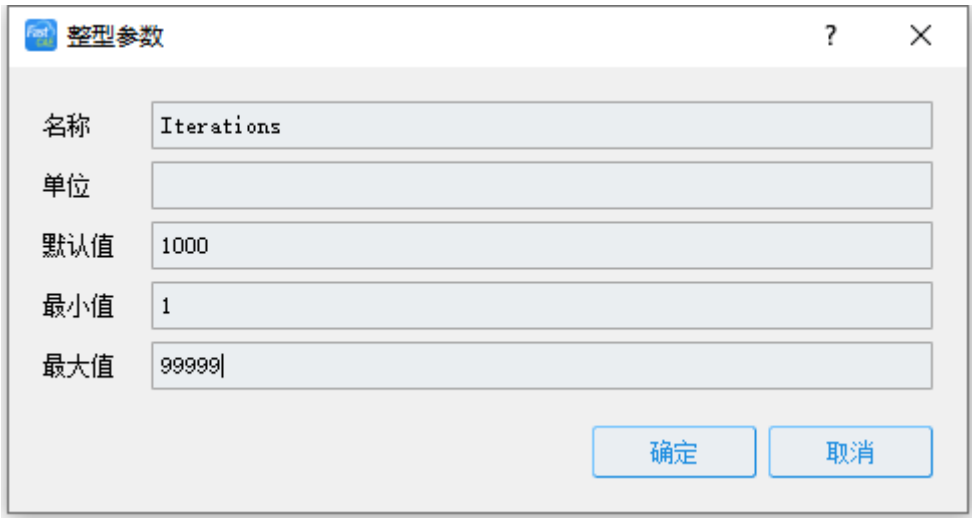


图 31 添加整形求解参数

31. 点击菜单中的【整形参数】菜单项，软件将弹出添加整形参数

界面。按照下图所示内容在添加整形参数界面添加信息，点击

确定 按键，完成 Iterations 整形参数的添加。

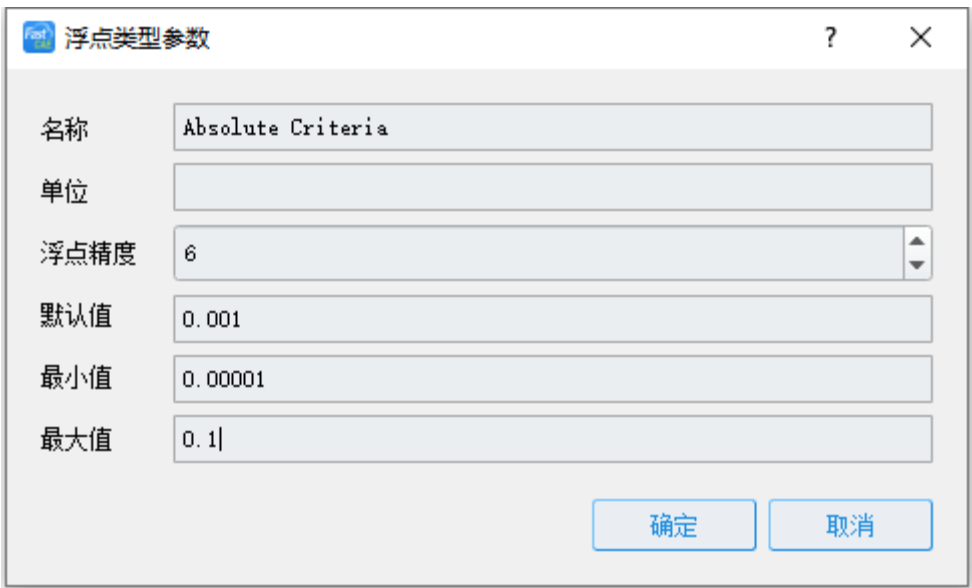


名称	Iterations
单位	
默认值	1000
最小值	1
最大值	99999

确定 取消

图 32 设置整形求解参数信息

32. 鼠标左键点击求解参数设置界面中的 **添加** 按键，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击 **确定** 按键，完成 Absolute Criteria 浮点参数的添加。



名称	Absolute Criteria
单位	
浮点精度	6
默认值	0.001
最小值	0.00001
最大值	0.1

确定 取消

图 33 设置浮点参数信息

33. 此时求解参数设置界面如下图所示，点击界面中的  按钮，完成求解参数的设置。

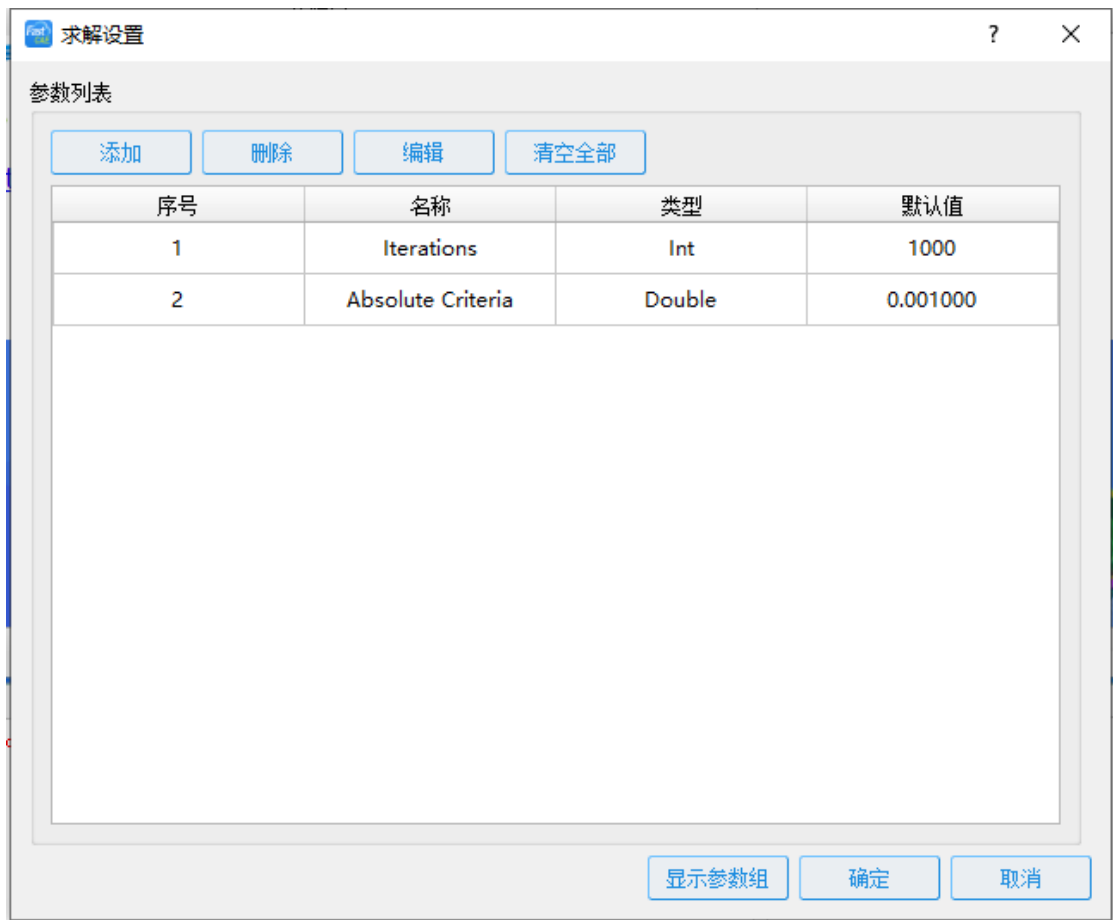


图 34 求解信息设置完成

34. Fluent 案例中不需求解过程中进行曲线的绘制。鼠标右键点击【控制面板】-【算例】节点下面的监视器节点，软件弹出【监视器】节点的右键菜单。

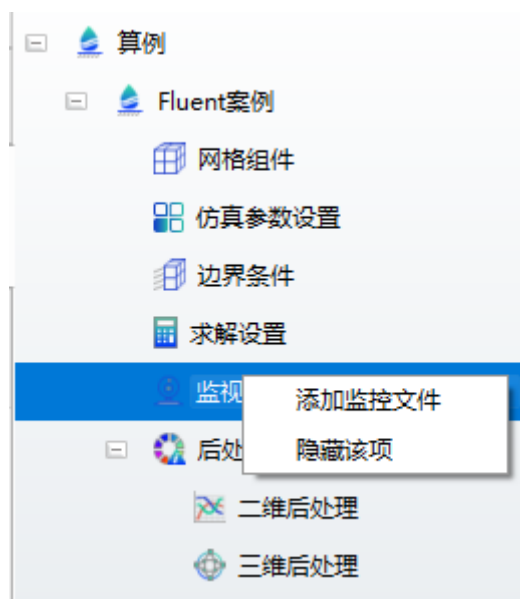


图 35 添加监控文件

35. 点击【监视器】节点右键菜单中的【隐藏该项】菜单项，将【监视器】节点隐藏。此时的【控制面板】-【算例】树如下图所示。

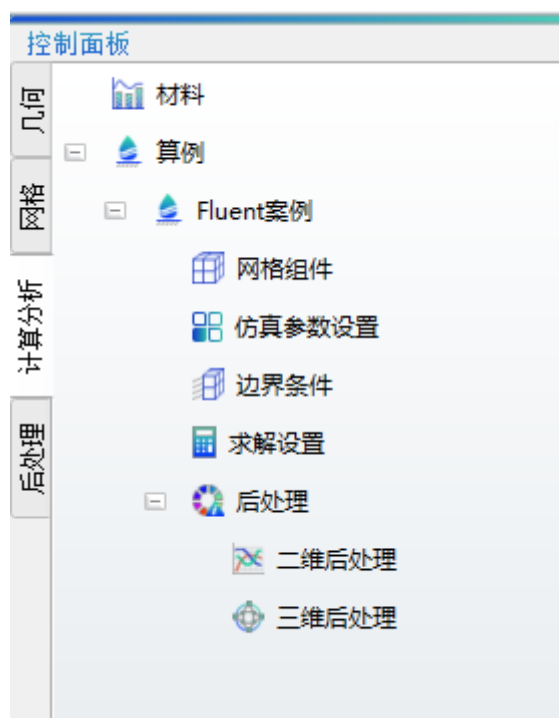


图 36 隐藏节点

36. Fluent 案例中不需要进行二维后处理。鼠标右键点击【控制面板】-【算例】节点下面的【二维后处理】节点，软件弹出【二维后

处理】节点的右键菜单。

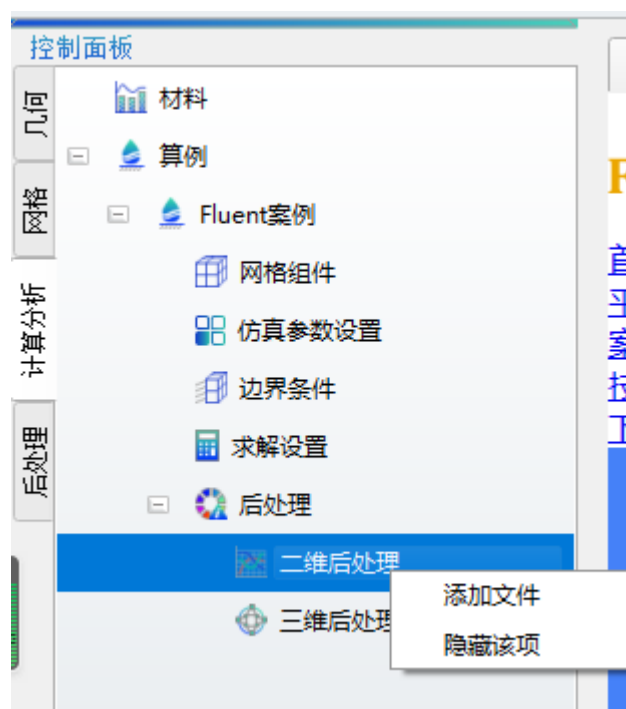


图 37 隐藏二维后处理节点

37. 点击【二维后处理】节点右键菜单中的【隐藏该项】菜单项，将【二维后处理】节点隐藏。此时的【控制面板】-【算例】树如下图所示。

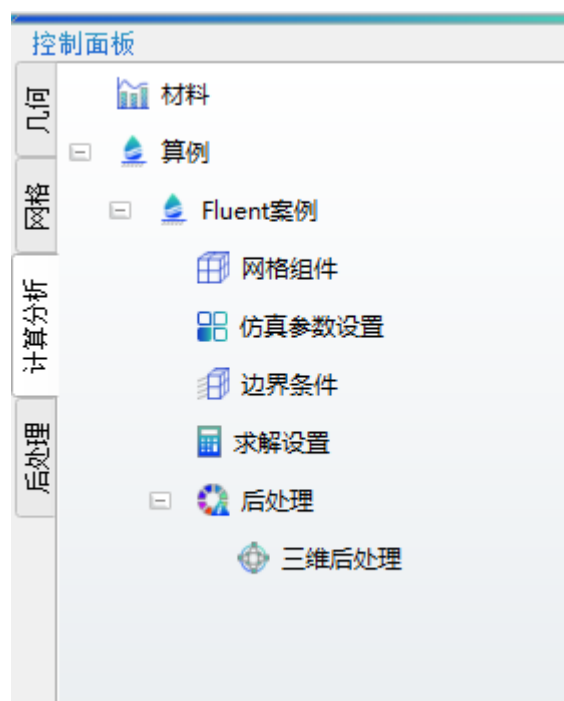


图 38 隐藏节点后的模型树

38. Fluent 案例具有三维后处理功能，因此我们需要对三维后处理进行设置。鼠标左键选中【控制面板】-【算例】下的【三维后处理】节点，点击鼠标右键，软件将弹出【三维后处理】节点右键菜单。

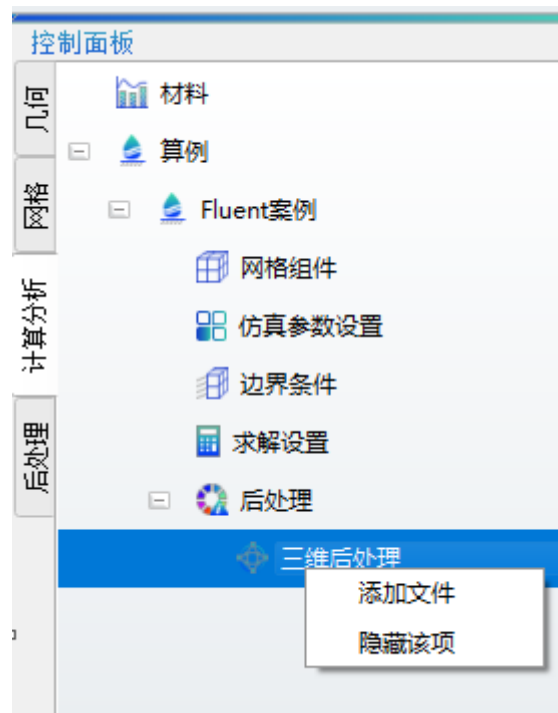


图 39 三维后处理添加文件

39. 点击【三维后处理】节点右键菜单中的【添加文件】菜单项，软件将弹出添加三维后处理文件名称的对话框。在对话框中我们输入进行三维后处理的文件的名称，如下图所示。点击  按钮完成名为 Result 的三维后处理文件的添加。下拉框中的格式我们选择.cgns。



图 40 设置三维后处理文件名称

40. 此时【控制面板】-【算例】的整个算例树如下图所示。

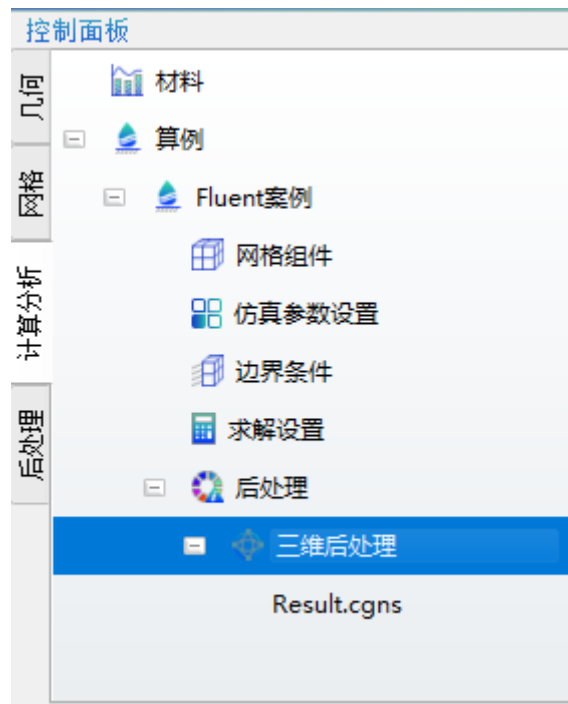


图 41 设置三维后处理文件名称后的模型树

41. 鼠标左键双击刚添加的【Result.cgns】节点，软件将弹出三维后处理【Result.cgns】节点绘图设置对话框。

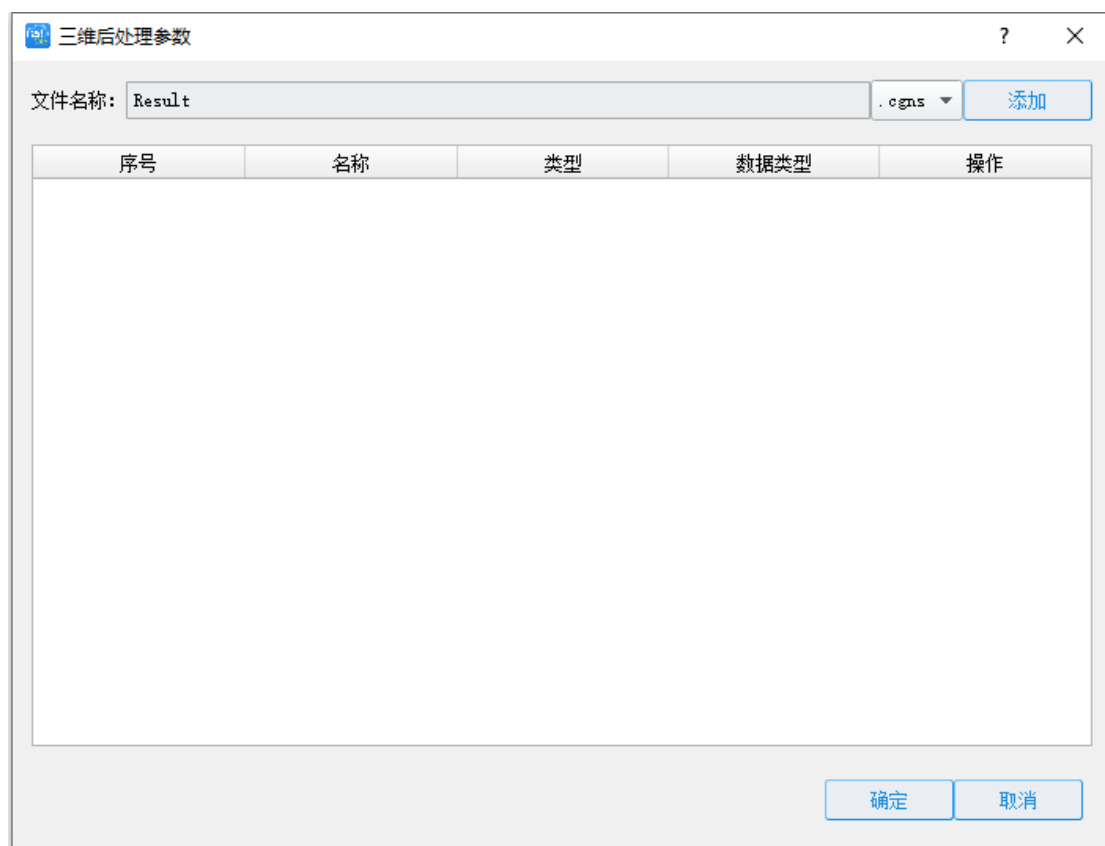


图 42 三维后处理文件信息设定

42. 由于 Result.cgns 文件中存有位于节点上的名为 Pressure 的标量值，因此我们点击【Result.cgns】节点上的 **添加** 按键，并按照下图填写勾选，完成后点击界面中的 **确定** 按键完成 Result.cgns 文件中位于节点上的名为 Pressure 的标量信息的添加。

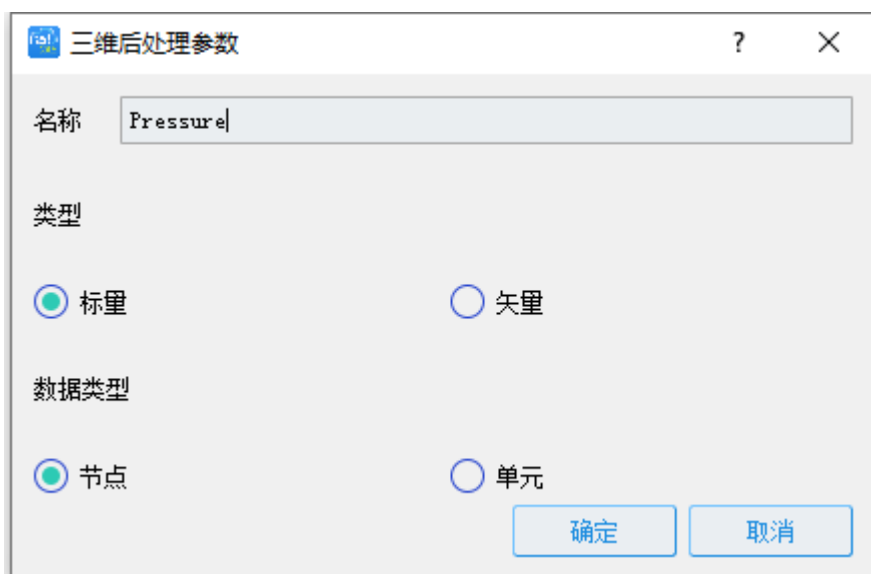


图 43 添加 Pressure 节点标量三维后处理参数

43. 由于 Result.cgns 文件中存有位于节点上的名为 PressureDynamic 的标量值，因此我们点击【Result.cgns】节点上的  按钮，并按照下图填写勾选，完成后点击界面中的  按钮完成 Result.cgns 文件中位于节点上的名为 PressureDynamic 的标量信息的添加。

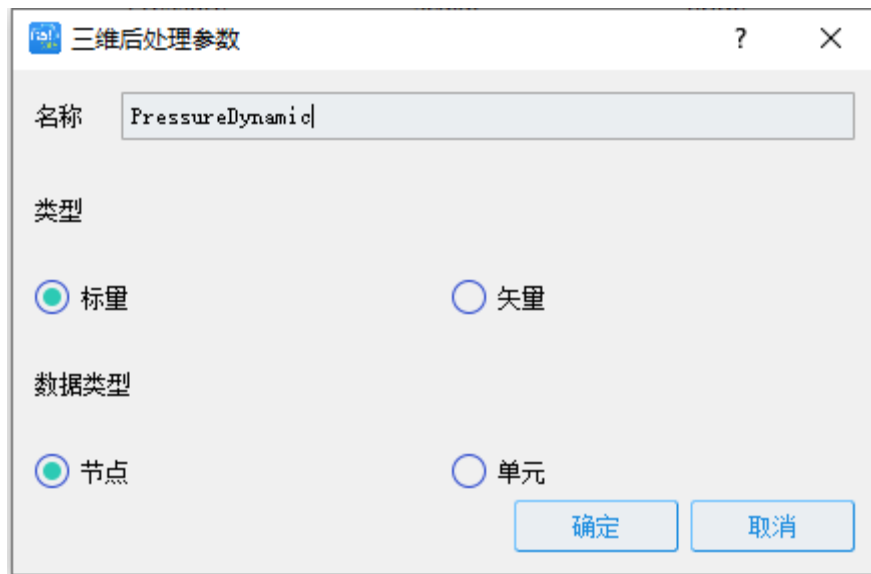


图 44 添加 PressureDynamic 节点标量三维后处理参数

44. 由于 Result.cgns 文件中存有位于节点上的名为 PressureStagnation 的标量值，因此我们点击【Result.cgns】节点上的  按钮，并按照下图填写勾选，完成后点击界面中的  按钮完成 Result.cgns 文件中位于节点上的名为 PressureStagnation 的标量信息的添加。

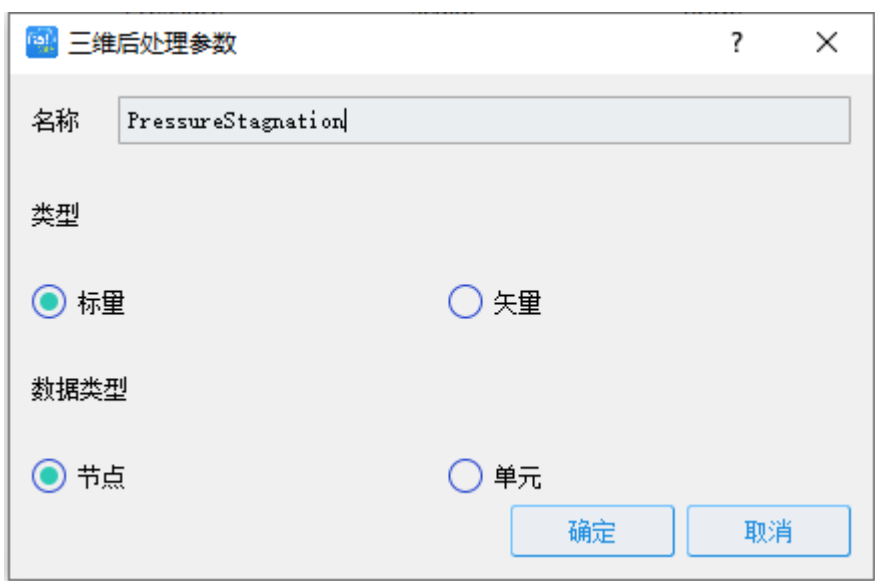


图 45 添加 PressureStagnation 节点标量三维后处理参数

45. 此时，三维后处理【Result.cgns】节点绘图设置对话框如下图所示。我们点击界面中的  按钮完成三维后处理【Result.cgns】节点的绘图信息设置。

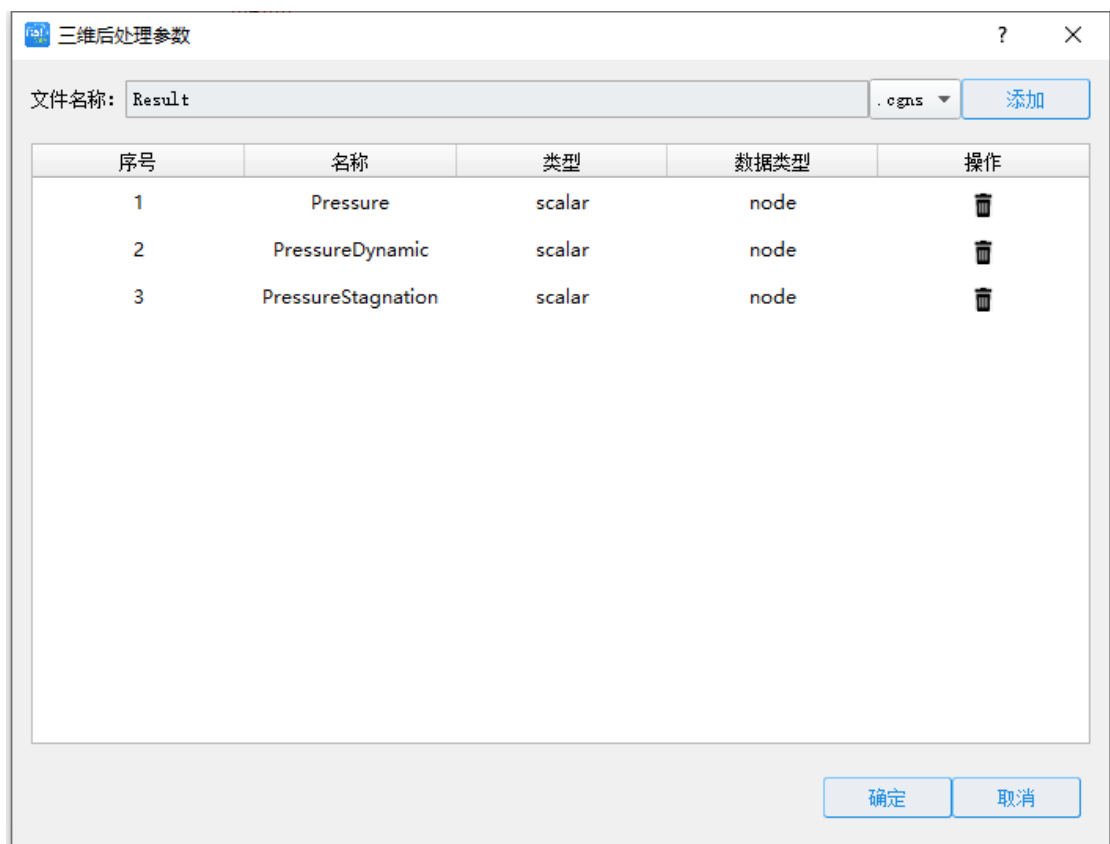


图 46 三维参数后处理信息设置完成

46. Fluent 案例求解器设置，求解器设置不需要在 PluginCustomizer.dll 插件中，即定制模式下进行设置。用户点击菜单栏【定制】-【完成定制】，将 FastCAE 2.0 软件切换至常规模式，点击菜单栏【求解器】-【求解器管理】项，软件将弹出求解器管理界面。



图 47 求解器管理

47. 点击求解器管理界面的 **添加** 按键，软件将弹出添加求解器界面，按照下图所示添加求解器信息。需要注意的是这里的求解器路径要选择 **Fluent.exe** 可执行文件所在的文件路径。点击界面中的 **确认** 按键完成求解器的添加。



图 48 编辑求解器

需要注意的是启动参数中的`%modelpath%`代表用户创建算例后算例路径。在我们开发的插件中会把替换完的 `jou` 脚本取名为 `script.jou` 并拷贝到算例路径中，所以启动参数处如此填写。注意：如果我们需要 `Fluent` 启动后再后台运行，则启动参数处我们可以添加`-hidden` 选项进行运行，此处没有`-hidden` 是为了方便我们进行调试。添加`-hidden` 后启动参数为 `3d -hidden -i %modelpath%/script.jou`。

48. 此时的求解器管理界面如下图所示，点击界面右上角的  按钮，关闭界面完成求解器的设置。

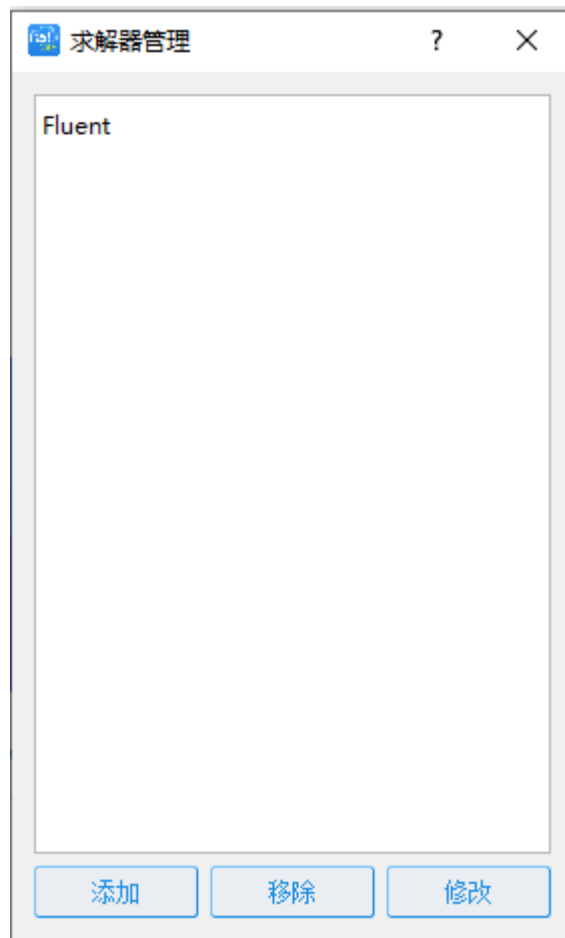


图 49 添加求解器完成

4.2 Fluent 插件开发

- 插件通用部分代码修改。

我们的插件继承自插件基类 `PluginBase`，因此基类的纯虚函数需要在我们的插件类中实现。这里我们把插件类名字命名为 `PluginFluent`。此时类所在的头文件 `pluginFluent.h` 内容如下：

```
namespace Plugins
{
    class _FLUENTPLUGINAPI FluentPlugin : public PluginBase
    {
        Q_OBJECT
    }
```

```

public:
    FluentPlugin(GUI::MainWindow* m);
    ~FluentPlugin();
    //加载插件
    bool install() override;
    //卸载插件
    bool uninstall() override;
private:
    GUI::MainWindow* _mainwindow{};
};
}

extern "C"
{
    void _FLUENTPLUGINAPI Register(GUI::MainWindow* m,
    QList<Plugins::PluginBase*>* plugs);
}

```

此时类所在的 CPP 文件 `pluginFluent.cpp` 内容如下：

```

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{
    Plugins::PluginBase* p = new Plugins::FluentPlugin(m);
    ps->append(p);
}

namespace Plugins
{
    FluentPlugin::FluentPlugin(GUI::MainWindow* m)
    {
        _mainwindow = m;
    }
}

```

```

FluentPlugin::~~FluentPlugin()
{
}

bool FluentPlugin::install()
{
    return true;
}

bool FluentPlugin::uninstall()
{
    return true;
}
}

```

在基类 `PluginBase` 中有一个 `Protected` 成员变量 `QString _describe`，它是用于描述当前插件信息的字符串变量。这里我们在 `FluentPlugin` 类的构造函数中对其赋值。

```
_describe = "Fluent Function Plugin";
```

- 插件需要完成函数选择。

当前 `FastCAE2.0` 插件支持的输入输出扩展函数包括：

```

//注册写出的后缀与方法
static void RegisterInputFile(QString suffix, WRITEINPFILE fun);

//注册结果文件转换方法
static void RegisterOutputTransfer(QString name, TRANSFEROUTFILE fun);

//注册网格文件导入的方法
static void RegisterMeshImporter(QString suffix, IMPORTMESHFUN fun);

//注册网格导出的方法
static void RegisterMeshExporter(QString suffix, EXPORTMESHFUN fun);

```

需要注意的是 `RegisterInputFile` 函数注册完成后，当在求解器

设置成我们所扩展的格式时，该函数会在**求解器求解之前**进行调用。而 `RegisterOutputTransfer` 函数注册完成后，会在**完成求解器求解生成结果文件后**调用。`RegisterMeshImporter` 和 `RegisterMeshExporter` 方法是为新格式的网格文件导入导出使用的函数。

根据我们的需求我们需要在**求解器求解之前**完成 Msh 格式网格文件的边界条件按照用户设置进行替换及 jou 脚本中参数替换。因此我们选择 `RegisterInputFile` 方法作为我们拓展的方法。因此我们在头文件中 C 函数部分添加：

```
///<MG 求解器求解之前的工作  
bool _FLUENTPLUGINAPI BeforeCalcFunc(QString path, ModelData::ModelDataBase* d);
```

在 CPP 文件中添加：

```
bool _FLUENTPLUGINAPI BeforeCalcFunc(QString path, ModelData::ModelDataBase* d)  
{  
    return true;  
}
```

并在 `BeforeCalcFunc` 函数中添加相应的功能代码，具体代码可以参见本段落最终头文件和 CPP 文件全部内容部分。

● 插件功能开发。

插件最终头文件内容：

```
#ifndef _MODELPLUGIN_H_  
#define _MODELPLUGIN_H_  
#include "FluentPluginAPI.h"  
#include "PluginManager/pluginBase.h"  
#include "DataProperty/modelTreeItemType.h"  
namespace GUI  
{
```



```

        class MainWindow;
    }
namespace MainWidget
{
    class PreWindow;
}
namespace ModelData
{
    class ModelDataBase;
}
namespace ProjectTree
{
    class ProjectTreeBase;
}
namespace Material
{
    class Material;
}
namespace Plugins
{
    class _FLUENTPLUGINAPI FluentPlugin : public PluginBase
    {
        Q_OBJECT
    public:
        FluentPlugin(GUI::MainWindow* m);
        ~FluentPlugin();
        //加载插件
        bool install() override;
        //卸载插件
        bool uninstall() override;

    private:
        GUI::MainWindow* _mainwindow{};
    };
}
extern "C"
{
    void _FLUENTPLUGINAPI Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>*
    plugs);
    ///<MG 求解器求解之前的工作
    bool _FLUENTPLUGINAPI BeforeCalcFunc(QString path, ModelData::ModelDataBase* d);
}
#endif

```

插件最终 CPP 文件内容:

```

#include "pluginFluent.h"
#include "mainWindow/mainWindow.h"
#include <QString>
#include "ModelData/modelDataFactory.h"
#include "ModelData/modelDataSingleton.h"
#include "ModelData/modelDataBase.h"
#include "MainWidgets/ProjectTreeFactory.h"
#include "IO/IOConfig.h"
#include "IO/TemplateReplacer.h"
#include "MeshData/meshSingleton.h"
#include "MeshData/meshSet.h"
#include "MeshData/meshKernal.h"
#include "BCBase/BCBase.h"
#include "ConfigOptions/ConfigOptions.h"
#include "ConfigOptions/SolverInfo.h"
#include <QDebug>
#include <QFile>
#include <QApplication>
#include <QDir>
void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{
    Plugins::PluginBase* p = new Plugins::FluentPlugin(m);
    ps->append(p);
}
namespace Plugins
{
    FluentPlugin::FluentPlugin(GUI::MainWindow* m)
    {
        _mainwindow = m;
        _describe = "Fluent Function Plugin";
    }
    FluentPlugin::~~FluentPlugin()
    {
    }
    bool FluentPlugin::install()
    {
        IO::IOConfigure::RegisterInputFile("Fluent", BeforeCalcFunc);
        return true;
    }
    bool FluentPlugin::uninstall()
    {
        IO::IOConfigure::RemoveInputFile("Fluent");
        return true;
    }
}

```

```

}

QString BCType2QString(BCBase::BCType type)
{
    if (type == BCBase::Inlet)
        return "velocity-inlet";
    else if (type == BCBase::Outlet)
        return "outflow";
    else if (type == BCBase::Wall)
        return "wall";
}

struct BCExchange
{
    QString bcName;
    BCBase::BCType toType;
};

bool ReplaceMeshFileBCType(const QString & fileName, QVector<BCExchange> exchangInfos)
{
    QFile file(fileName);
    if (!file.exists()) { return false; }

    QString head = QString("(45 ");
    QString tail = QString("))\r\n");

    if (file.open(QIODevice::ReadWrite))
    {
        file.seek(file.size() - 1000);

        QString toContent;
        while (!file.atEnd())
        {
            QString lineContent = file.readLine();
            if (lineContent.size() < 6)
            {
                toContent += lineContent;
                continue;
            }
            if (lineContent.left(head.size()) == head && lineContent.right(tail.size())
== tail)
            {
                QStringList infos = lineContent.mid(5, lineContent.size() - 5 -
6).split(" ", QString::SkipEmptyParts);
                if (infos.size() == 3)

```

```

        {
            QString id = infos[0], type = infos[1], name = infos[2];

            bool needReplace = false;
            for (BCExchange one : exchangInfos)
            {
                if (name == one.bcName)
                {
                    QString setName = "(45 (" + id + " " +
BCType2QString(one.toType) + " " + name + ")())\r\n";
                    toContent += setName;
                    needReplace = true;
                }
            }
            if (!needReplace)
                toContent += lineContent;
        }
    }
    else
        toContent += lineContent;
}

file.seek(0);
QString totalContent = file.read(file.size() - 1000) + toContent;
file.seek(0);
file.resize(totalContent.size());
file.write(totalContent.toStdString().c_str());
return true;
}
else
    return false;
}

```

```

bool BCOperation(ModelData::ModelDataBase* d, QString & meshFilePath)
{
    int nBC = d->getBCCount();
    if (nBC == 0) { return false; }
    QMap<QString, QVector<BCExchange>> bcExchangeInfos;
    for (int iBC = 0; iBC < nBC; ++iBC)
    {
        BCBase::BCBase * pBC = d->getBCAt(iBC);
        if (!pBC) { continue; }
        ///<MG 获取边界条件类型，及对应边界条件网格组件的名称
        BCBase::BCType bcType = pBC->getType();
        int meshSetID = pBC->getMeshSetID();
    }
}

```

```

        MeshData::MeshSet * meshSet =
MeshData::MeshData::getInstance()->getMeshSetByID(meshSetID);
        if (!meshSet){ continue; }
        QString meshSetName = meshSet->getName();
        QList<int> meshKernalIDs = meshSet->getKernals();
        for (int KernalID : meshKernalIDs)
        {
            MeshData::MeshKernal * pKernal =
MeshData::MeshData::getInstance()->getKernalByID(KernalID);
            if (!pKernal){ continue; }
            QString filePath = pKernal->getPath();
            ///

```

```

"/../Template/template.jou";
QFile file(templatePath);
QString toPath = path + "/script.jou";
if (file.exists(toPath)) { file.remove(toPath); }
QString toOutputPath = path + "/Result/Result.cgns";
file.copy(templatePath, toPath);

///

```

```

    if (!JournalOperation(path, d, meshFilePath)){ return false; }
    return true;
}

```

其中 **BCOperation** 函数用于将传入的 **msh** 文件按照当前 FastCAE2.0 中设置的边界条件进行重写。**JournalOperation** 函数用于 Jou 脚本中参数的替换。具有 C++基础的用户可自行研读这部分代码内容，可以发现 jou 脚本替换是通过替换脚本中**%参数名称%**为参数值来实现的。

4.3 脚本修改

Fluent 的 jou 脚本采用了 SScheme 语言，我们可以看到在我们使用的录制 jou 脚本中：

```

1 (cx-gui-do cx-activate-item "MenuBar*ReadSubMenu*Mesh...")
2 (cx-gui-do cx-set-file-dialog-entries "Select File" '("E:/FluentCaseFiles/0814.msh") "Mesh Files (*.msh;*.MSH)")
3 (cx-use-window-id 1)
4 (cx-set-camera-relative '(-1.01367 1.08424 -0.414614) ' (0.0 0.0) ' (0.0506475 -0.0791588 -0.386639) 0.0)
5 (cx-gui-do cx-activate-item "Ribbon*Frame1*Frame2 (Setting Up Domain) *Table1*Table3 (Mesh) *PushButton1 (Display)")
6 (cx-gui-do cx-activate-item "Mesh.Display*PanelButtons*PushButton1 (OK)")
7 (cx-gui-do cx-activate-item "Mesh.Display*PanelButtons*PushButton2 (Cancel)")
8
9 (cx-gui-do cx-activate-item "General*Table1*ButtonBox1 (Mesh) *PushButton3 (Check)")
10 (cx-gui-do cx-activate-item "General*Table1*ButtonBox1 (Mesh) *PushButton1 (Scale)")
11 (cx-gui-do cx-set-list-selections "Scale.Mesh*Table1*DropDownList3 (View.Length.Unit.In)" '(.2))
12 (cx-gui-do cx-activate-item "Scale.Mesh*Table1*DropDownList3 (View.Length.Unit.In)")
13 (cx-gui-do cx-set-list-selections "Scale.Mesh*Table1*Table2 (Scaling) *DropDownList2 (Mesh.Was.Created.In)" '(.3))
14 (cx-gui-do cx-activate-item "Scale.Mesh*Table1*Table2 (Scaling) *DropDownList2 (Mesh.Was.Created.In)")
15 (cx-gui-do cx-activate-item "Scale.Mesh*PanelButtons*PushButton1 (Close)")
16 (cx-gui-do cx-set-list-tree-selections "NavigationPane*List_Tree1" (list "Setup|Models"))
17 (cx-gui-do cx-set-list-tree-selections "NavigationPane*List_Tree1" (list "Setup|Models"))
18 (cx-gui-do cx-activate-item "NavigationPane*List_Tree1")
19 (cx-gui-do cx-activate-item "Models*Table1*PushButton2 (Edit)")
20 (cx-gui-do cx-set-toggle-button2 "Multiphase.Model*Table1*ToggleBox1 (Model) *Mixture" #t)
21 (cx-gui-do cx-activate-item "Multiphase.Model*Table1*ToggleBox1 (Model) *Mixture")
22 (cx-gui-do cx-activate-item "Multiphase.Model*PanelButtons*PushButton1 (OK)")
23 (cx-gui-do cx-set-list-selections "Models*Table1*List1 (Models)" '(.2))
24 (cx-gui-do cx-activate-item "Models*Table1*List1 (Models)")
25 (cx-gui-do cx-activate-item "Models*Table1*PushButton2 (Edit)")
26 (cx-gui-do cx-set-toggle-button2 "Viscous.Model*Table1*ToggleBox1 (Model) *k-omega (2.eqn)" #t)
27 (cx-gui-do cx-activate-item "Viscous.Model*Table1*ToggleBox1 (Model) *k-omega (2.eqn)")
28 (cx-gui-do cx-activate-item "Viscous.Model*PanelButtons*PushButton1 (OK)")
29 (cx-gui-do cx-set-list-tree-selections "NavigationPane*List_Tree1" (list "Setup|Materials"))
30 (cx-gui-do cx-set-list-tree-selections "NavigationPane*List_Tree1" (list "Setup|Materials"))
31 (cx-gui-do cx-activate-item "NavigationPane*List_Tree1")
32 (cx-gui-do cx-set-list-tree-selections "NavigationPane*List_Tree1" (list "Setup|Materials"))
33 (cx-gui-do cx-set-list-selections "Materials*Table1*List1 (Materials)" '(.0))

```

图 50 原始 jou 脚本文件

第 1、2 行表示打开 Fluent 菜单项 Read Mesh 读取 “E:/FluentCaseFiles/0814.msh” 网格文件。

第 26 、 27 行表示 Viscous Model 项勾选为 k-omega(2 eqn)。

由于我们需要与 FastCAE2.0 进行对接，将我们需要在 FastCAE2.0 中设置的参数在 jou 文件中进行替换，这里我们采用%参数名称%形式对 jou 脚本中的参数值进行替换，以第 26 、 27 行为例，我们要将第 26 、 27 行的值 k-omega(2 eqn)替换为%Viscous Model%，其中 Viscous Model 为该参数的名称，k-omega(2 eqn)为该参数当前的设置值。对导入网格的路径采用%MshPath%形式对 jou 脚本中的导入网格路径进行替换，对导出结果文件采用%OutputPath%形式对 jou 脚本中导出结果文件路径进行替换。

经过替换后的前 33 行内容如下图所示，整个替换前的脚本和替换后的脚本可以通过下载获取。用户也可以根据需求录制自己使用的脚本，并将结果文件和网格文件以及其他想要在 FastCAE2.0 集成软件中进行设置的参数进行替换，完成自己需要的 jou 脚本文件。

```
1 (cx-gui-do.cx-activate-item."MenuBar*ReadSubMenu*Mesh...")↓
2 (cx-gui-do.cx-set-file-dialog-entries."Select.File".'("%MshPath%")."Mesh.Files.(*.msh*.*.MSH*")')↓
3 (cx-use-window-id:1)↓
4 (cx-set-camera-relative.'(-1.01367.1.08424.-0.414614).'(0.0.0).'(0.0506475.-0.0791588.-0.386639).0.0)↓
5 (cx-gui-do.cx-activate-item."Ribbon*Frame1*Frame2(Setting.Up.Domain)*Table1*Table3(Mesh)*PushButton1(Display)")↓
6 (cx-gui-do.cx-activate-item."Mesh.Display*PanelButtons*PushButton1(OK)")↓
7 (cx-gui-do.cx-activate-item."Mesh.Display*PanelButtons*PushButton2(Cancel)")↓
8 ↓
9 (cx-gui-do.cx-activate-item."General*Table1*ButtonBox1(Mesh)*PushButton3(Check)")↓
10 (cx-gui-do.cx-activate-item."General*Table1*ButtonBox1(Mesh)*PushButton1(Scale)")↓
11 (cx-gui-do.cx-set-list-selections."Scale.Mesh*Table1*DropDownList3(View.Length.Unit.In)".'(.2)')↓
12 (cx-gui-do.cx-activate-item."Scale.Mesh*Table1*DropDownList3(View.Length.Unit.In)")↓
13 (cx-gui-do.cx-set-list-selections."Scale.Mesh*Table1*Table2(Scaling)*DropDownList2(Mesh.Was.Created.In)".'(.3)')↓
14 (cx-gui-do.cx-activate-item."Scale.Mesh*Table1*Table2(Scaling)*DropDownList2(Mesh.Was.Created.In)")↓
15 (cx-gui-do.cx-activate-item."Scale.Mesh*PanelButtons*PushButton1(Close)")↓
16 (cx-gui-do.cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Models"))↓
17 (cx-gui-do.cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Models"))↓
18 (cx-gui-do.cx-activate-item."NavigationPane*List_Tree1")↓
19 (cx-gui-do.cx-activate-item."Models*Table1*PushButton2(Edit)")↓
20 (cx-gui-do.cx-set-toggle-button2."Multiphase.Model*Table1*ToggleBox1(Model)*Mixture".#t)↓
21 (cx-gui-do.cx-activate-item."Multiphase.Model*Table1*ToggleBox1(Model)*Mixture")↓
22 (cx-gui-do.cx-activate-item."Multiphase.Model*PanelButtons*PushButton1(OK)")↓
23 (cx-gui-do.cx-set-list-selections."Models*Table1*List1(Models)".'(.2)')↓
24 (cx-gui-do.cx-activate-item."Models*Table1*List1(Models)")↓
25 (cx-gui-do.cx-activate-item."Models*Table1*PushButton2(Edit)")↓
26 (cx-gui-do.cx-set-toggle-button2."Viscous Model*Table1*ToggleBox1(Model)*%Viscous Model%".#t)↓
27 (cx-gui-do.cx-activate-item."Viscous Model*Table1*ToggleBox1(Model)*%Viscous Model%")↓
28 (cx-gui-do.cx-activate-item."Viscous Model*PanelButtons*PushButton1(OK)")↓
29 (cx-gui-do.cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Materials"))↓
30 (cx-gui-do.cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Materials"))↓
31 (cx-gui-do.cx-activate-item."NavigationPane*List_Tree1")↓
32 (cx-gui-do.cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Materials"))↓
33 (cx-gui-do.cx-set-list-selections."Materials*Table1*List1(Materials)".'(.0)')↓
34 (cx-gui-do.cx-activate-item."Materials*Table1*List1(Materials)")↓
```

图 51 替换后的 jou 脚本文件

在完成脚本修改后，我们找到 FastCAE 2.0 应用所在路径的上级目录下创建名为 **Template** 的文件夹，此时 **Template** 文件夹所在的文件夹如下图所示。

名称	修改日期	类型	大小
 bin	2020/2/12 15:29	文件夹	
 bin_d	2020/2/12 15:14	文件夹	
 ConfigFiles	2020/2/12 15:19	文件夹	
 Install	2020/2/13 14:13	文件夹	
 python37	2020/2/6 11:11	文件夹	
 Template	2020/2/13 14:35	文件夹	
 WorkingDir	2020/2/7 9:57	文件夹	

图 52 案例文件目录结构

我们将替换完的脚本文件命名为 **template.jou**，并将其拷贝到 **Template** 文件夹下。从 **Fluent** 插件开发中的源码部分我们可以看出，每次求解的时候我们会将 **Template** 文件夹下的 **template.jou** 文件拷贝到算例所在文件夹，重命名为 **script.jou**，并将脚本文件中的需要替换的参数通过程序完成替换。

4.4 软件使用

49. 此时我们已经完成了定制部分工作，我们将开始运行我们所做的程序进行求解工作。鼠标左键点击【控制面板】-【计算分析】-【算例】节点，单击鼠标右键软件将弹出【算例】节点的右键菜单。



图 53 创建算例

50. 点击【算例】节点弹出的右键菜单中的【创建算例】菜单项，软件会弹出创建算例对话框。这里我们使用默认的名称，直接点击  按钮完成算例的创建。如果有需要用户可以输入其他的算例名称。



图 54 填写算例名称

51. 此时算例下的树形结构如下图所示。点击【算例】下的【仿真参数设置】节点，我们可以看到仿真参数设置节点下的定制的参数及相应默认的参数值。用户可以根据需求，自行调整参数值。

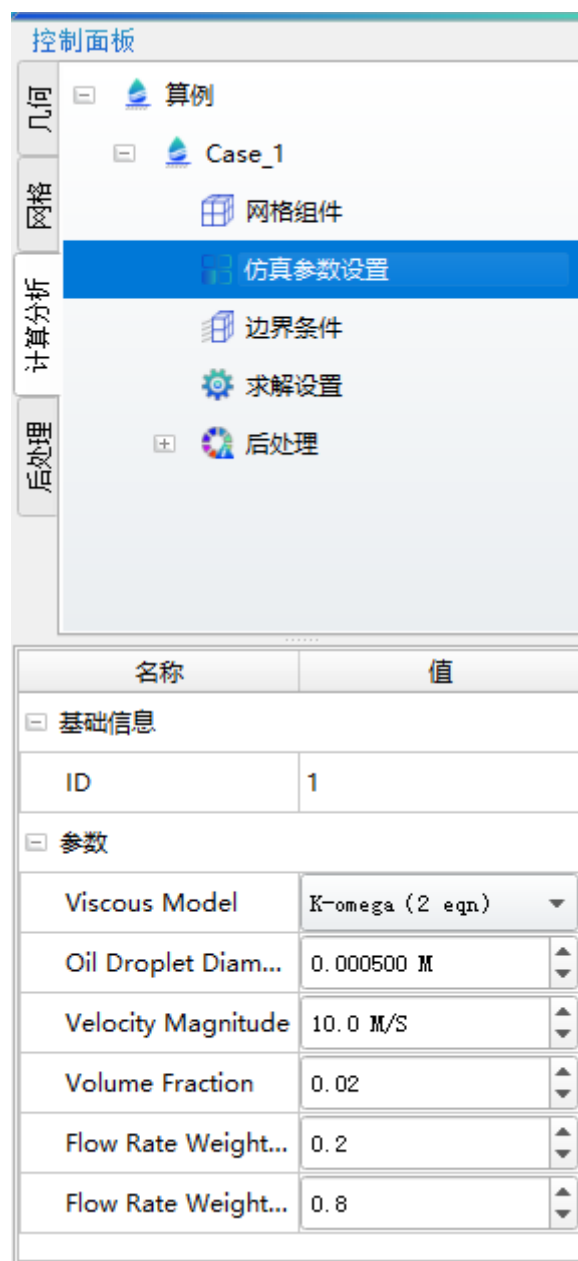


图 55 控制面板界面

52. 点击【算例】下的【求解设置】节点，我们可以看到求解设置节点下的定制的参数及相应默认的参数值。用户可以根据需求，自行调整参数值。



图 56 求解设置参数及其默认值

53. 导入网格。点击【控制面板】-【网格】节点，将控制面板切换到网格界面，鼠标左键点击【网格】节点，点击鼠标右键，软件将弹出【网格】节点右键菜单。



图 57 导入网格

54. 点击【网格】节点右键菜单中的【导入网格】菜单项，软件将弹出导入网格对话框。这里我们选择我们案例需要的 0814.msh 网格文件后，点击按键，软件将加载 0814.msh 文件。

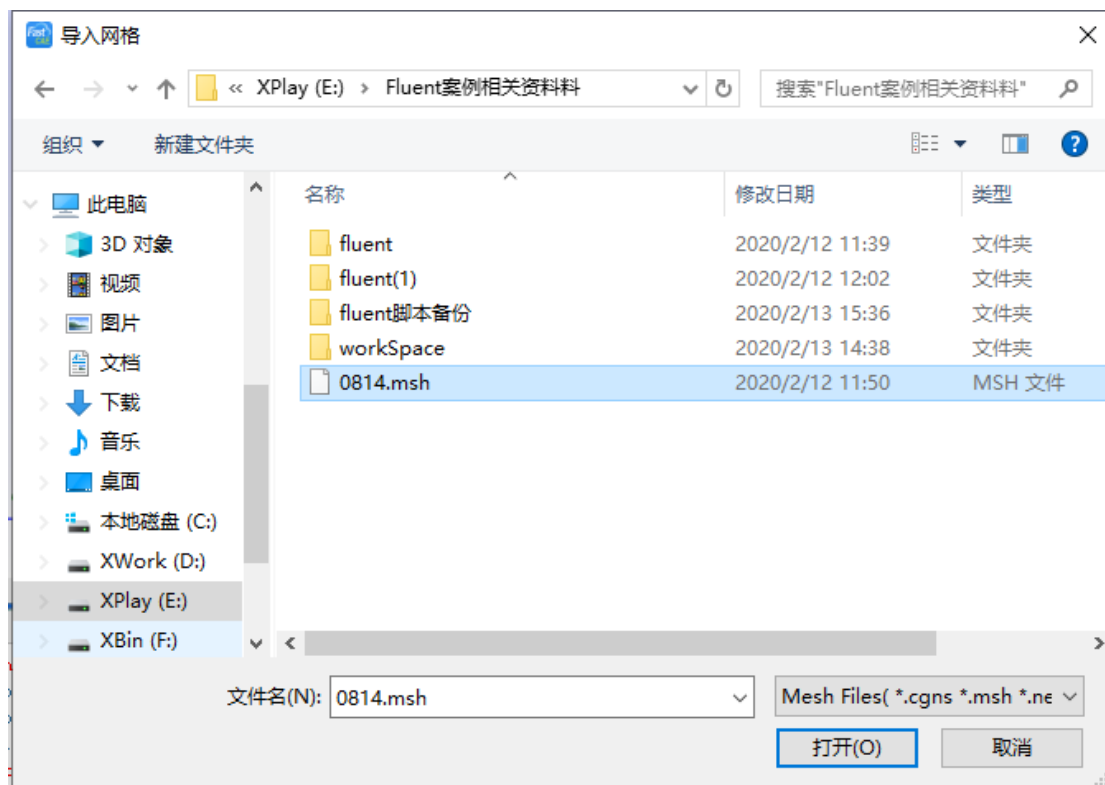


图 58 选择网格文件

55. 此时主界面中将在 0814.msh 文件，如下图所示。

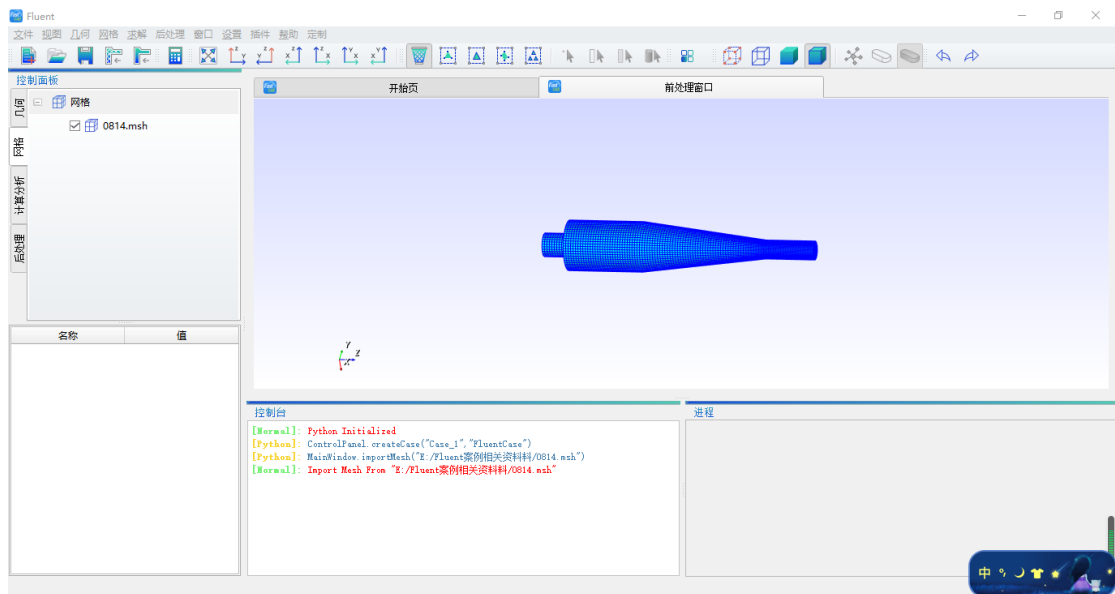


图 59 导入网格完成

56. 点击【控制面板】-【计算分析】将控制面板切换回算例所在分页。鼠标左键选中【算例】节点下的【网格组件】节点，点击鼠标右键，软件弹出【网格组件】节点右键菜单。

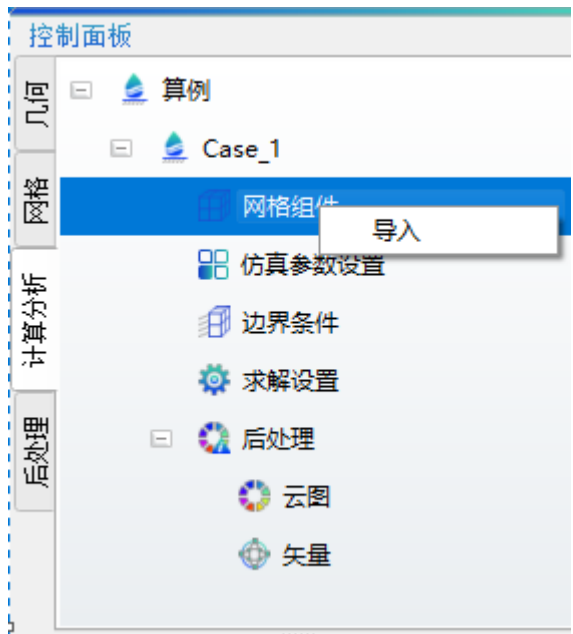


图 60 导入网格组件

57. 点击【网格组件】节点右键菜单中的【导入】菜单项。软件弹

出网格组件导入界面。

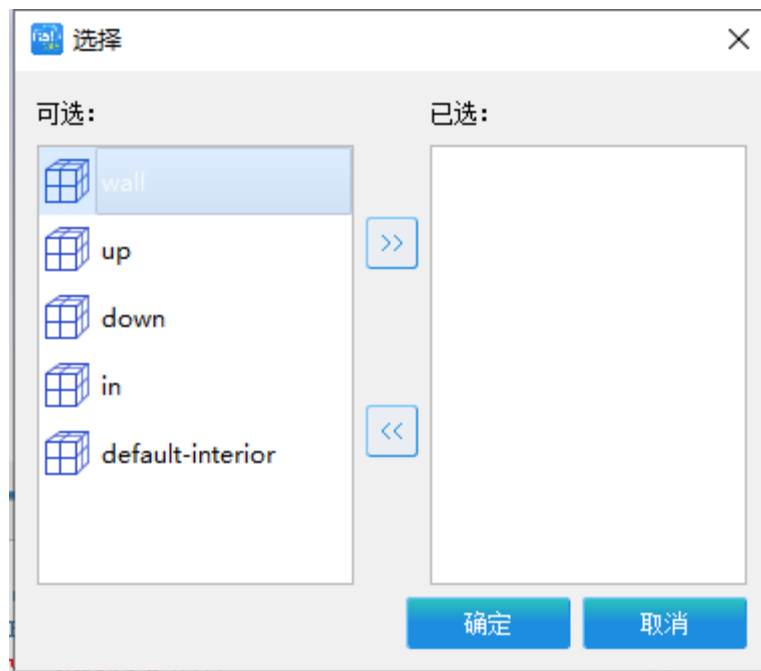


图 61 网格组件选择界面

58. 在网格组件导入窗体界面中，我们使用鼠标左键将可选组件【wall】、【up】、【down】、【in】、【default-interior】组件全部选中后点击  按钮，这样全部组件都会在右侧已选列表中，点击  按钮完成网格组件的导入。

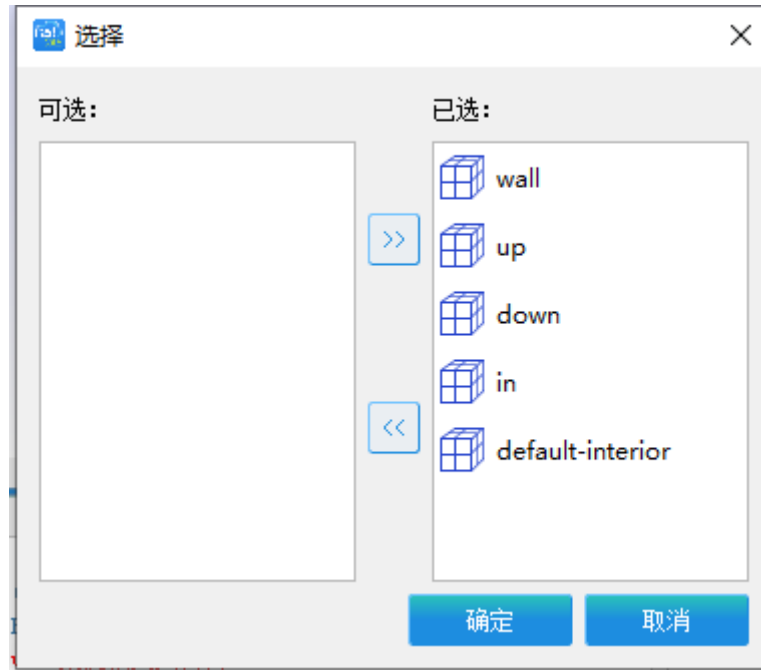


图 62 网格组件选择完成界面

59. 此时的控制面板如下图所示。



图 63 导入网格组件后的控制面板

60. 下面我们需要为不同的网格组件设置不同的边界条件，鼠标左键点击【算例】节点下的【边界条件】节点使其处于选中状态，点击鼠标右键，软件将弹出【边界条件】节点右键菜单。

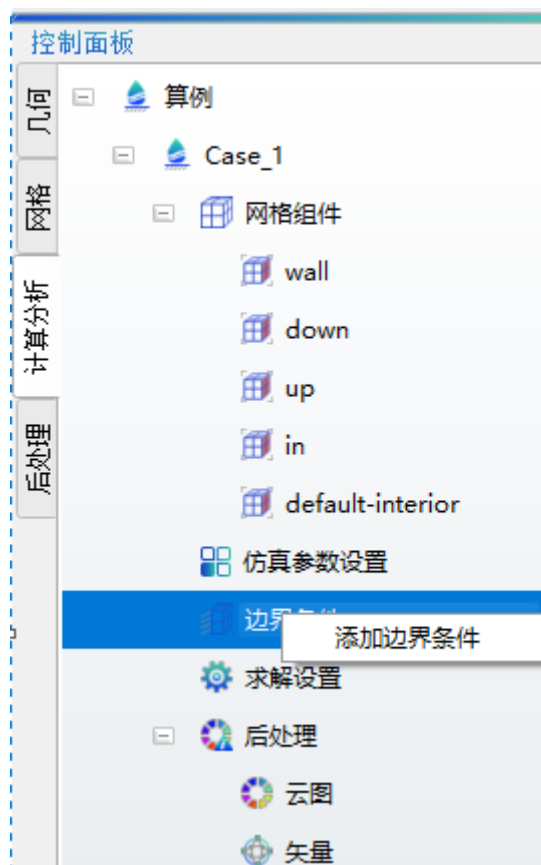


图 64 添加边界条件

61. 点击【边界条件】节点右键菜单中的【添加边界条件】菜单项，软件将弹出边界条件设置对话框。在对话框中，我们在网格组件选择为【wall】网格组件，类型选择为【Wall】壁面类型后，点击  按钮，这样就完成了【wall】网格组件的边界条件设定。

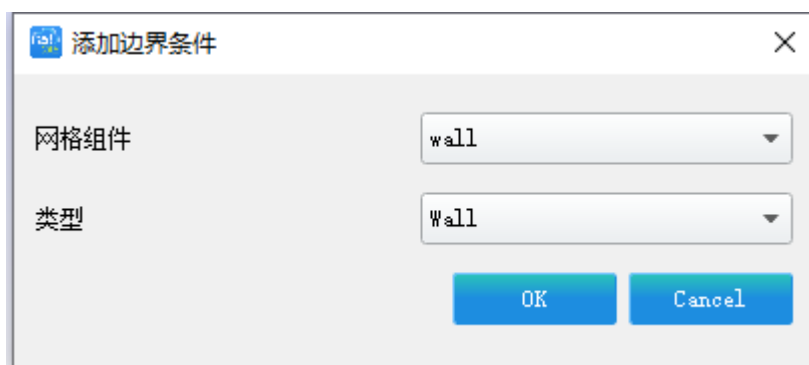


图 65 添加壁面边界条件

62. 按照同样方法将【down】网格组件设置为【Outlet】出口类型，将【up】网格组件设置为【Outlet】出口类型，将【in】网格组件设

置为【Inlet】入口类型，此时【控制面板】-【计算分析】中的算例模型树如下图所示。

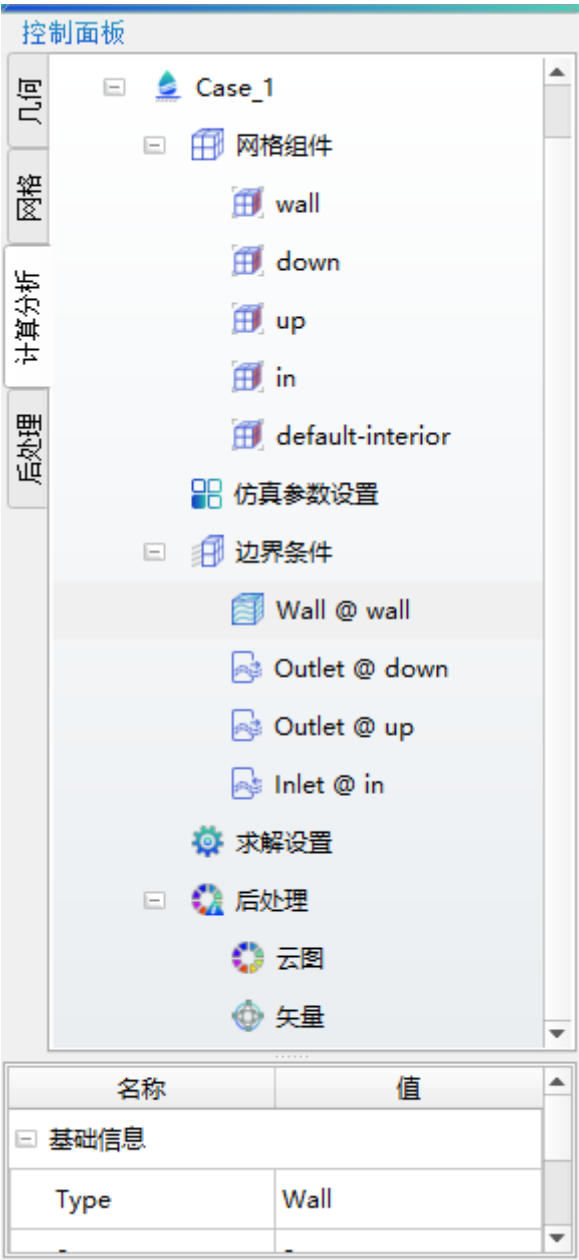


图 66 添加完边界条件的控制面板

63. 点击工具栏的  按键，软件将启动求解。在求解器完成后用户鼠标右键选择点击【后处理】节点下的【云图】节点，在弹出的右键菜单中选择【节点标量】-【Pressure】。

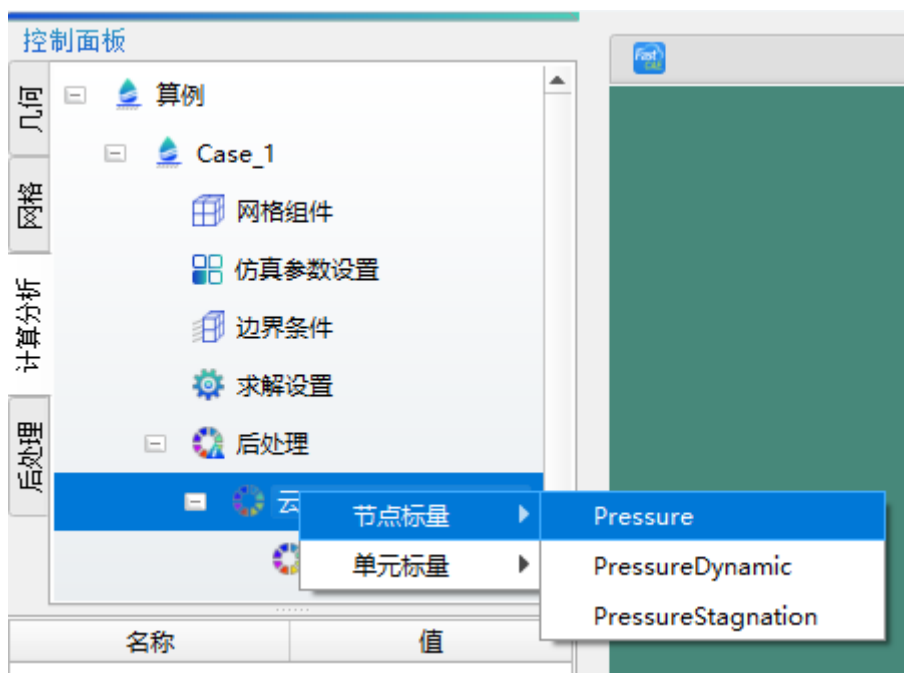


图 67 后处理压力云图操作

64. 完成上一步的操作后，软件后处理将访问我们算例下 Result/Result.cgns 结果文件，并使用 Pressure 标量进行云图绘制。结果如下图所示。

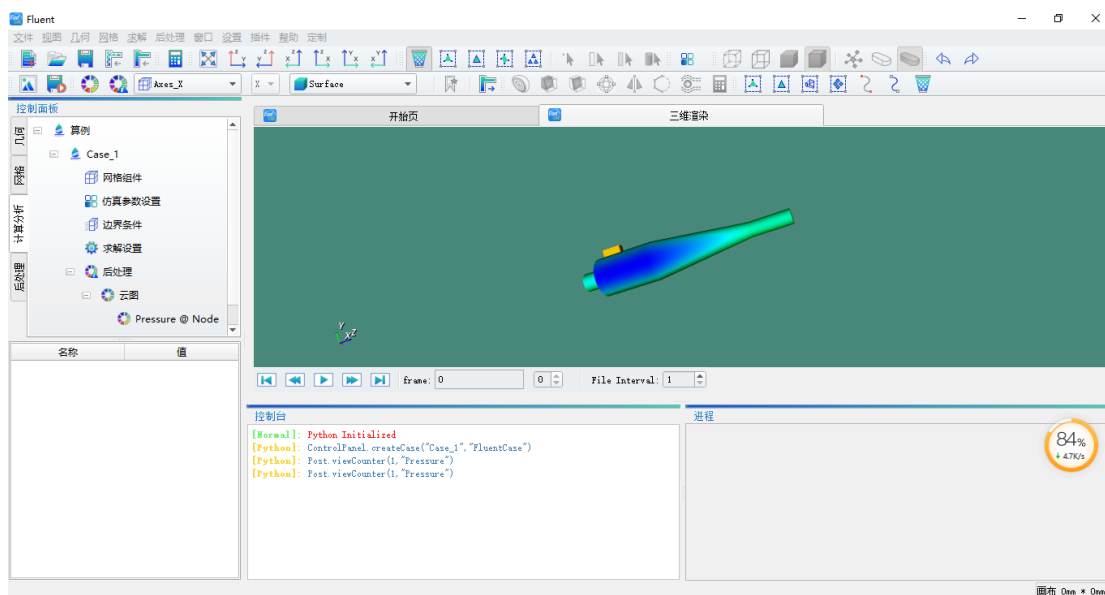


图 68 软件后处理效果